Design Document

for

Mess Automation System

Version 1.0

Prepared by

Group 10: **Group Name:** Byte Busters

B Mahath	240252	bmahath24@iitk.ac.in
Priyanshi Meena	240804	priyanshim24@iitk.ac.in
Rishith Jalagam	240484	rishithjs24@iitk.ac.in
Eranki Venkata Jhanani	240385	vjhanani24@iitk.ac.in
Basavakiran Shivanand Deshmukh	240261	bshivanand24@iitk.ac.in
Inapakurti Rajesh	240463	rajeshr24@iitk.ac.in
Parth Parashar	240735	parthp24@iitk.ac.in
Shreyansh Dewangan	230987	shreyanshd23@iitk.ac.in
Rohan Kumar	230865	rohank23@iitk.ac.in
Siddhant Singh	231004	siddhant23@iitk.ac.in

Course: CS253

Mentor TA: *Ujjwal Kajal*

Date: 6th February 2026

CONTENTS II

REVISIONS III

1 CONTEXT DESIGN 1

1.1 CONTEXT MODEL 1
1.2 HUMAN INTERFACE DESIGN 2

2 ARCHITECTURE DESIGN 11

3 OBJECT-ORIENTED DESIGN 15

3.1 USE CASE DIAGRAM 15
3.2 CLASS DIAGRAM 21
3.3 SEQUENCE DIAGRAM 22
3.4 STATE DIAGRAM 37

4 PROJECT PLAN 38

APPENDIX A - GROUP LOG 40

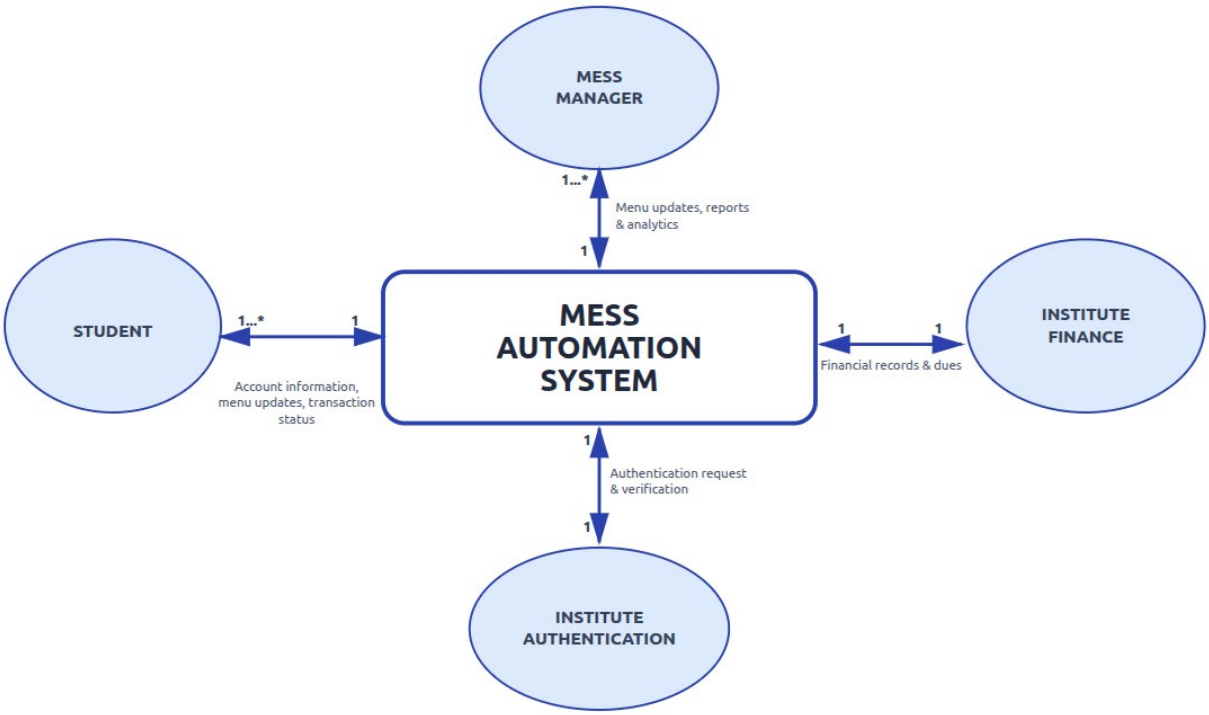
Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Initial Draft (v1.0)	B Mahath Priyanshi Meena Shreyansh Basavakiran Rishit Rajesh Parth Jhanani Rohan Siddhant	Initiated the Document and added all necessary details.	06/02/26

1 Context Design

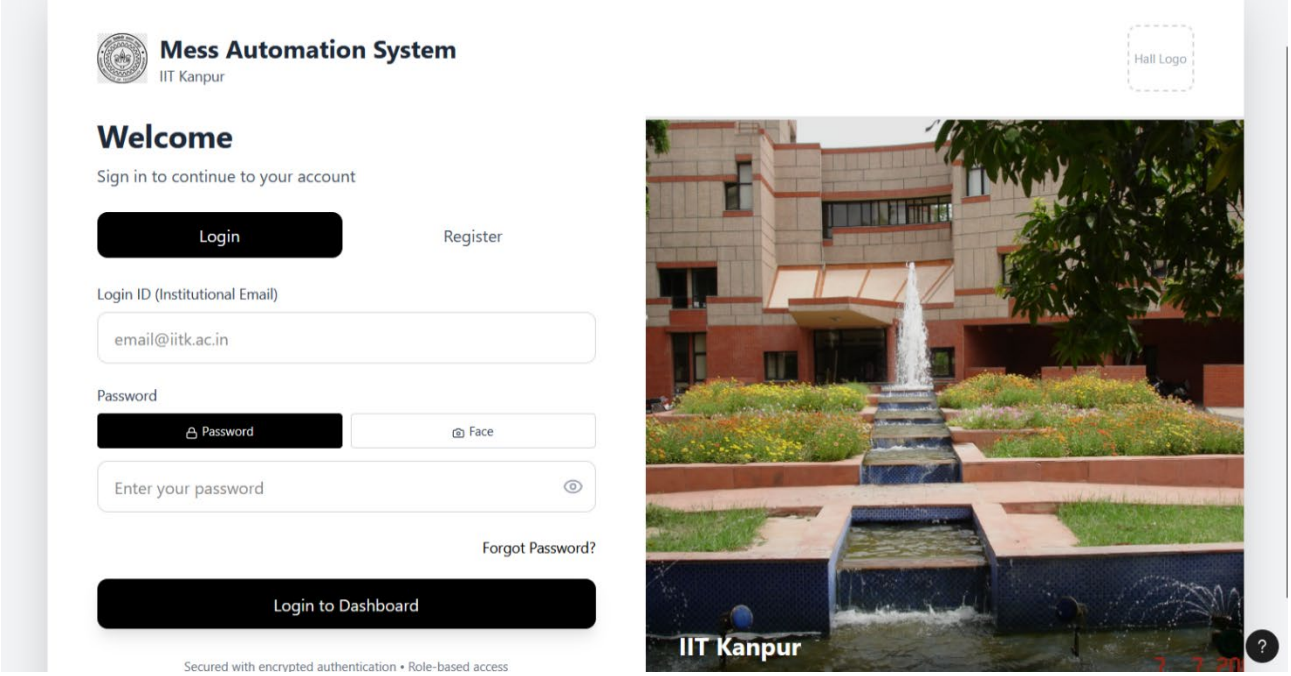
1.1 Context Model

1. The context model represents the Mess Automation System as a single centralised system and illustrates its interaction with external entities. It defines the system boundary and shows the flow of information between the system and its users and supporting institutional services.
2. Students interact with the system to log in, view menus, vote for dishes, purchase extra food items, and provide feedback. In return, the system provides account information, menu updates, and transaction status. Multiple students (1..*) interact with one system.
3. Mess Managers use the system to update menus, manage inventory, and control voting processes. The system provides reports and analytics to support operational decision-making. One or more mess managers (1..*) manage the system.
4. The Institute Finance department receives financial records and monthly dues from the system for billing and accounting purposes. A single finance entity (1) interacts with the system.
5. The Institute Authentication service handles secure user verification by receiving authentication requests and returning verification responses. A single authentication service (1) is used.
6. Overall, the context model clearly establishes system scope, external dependencies, and high-level information exchange, serving as a foundation for detailed system design.

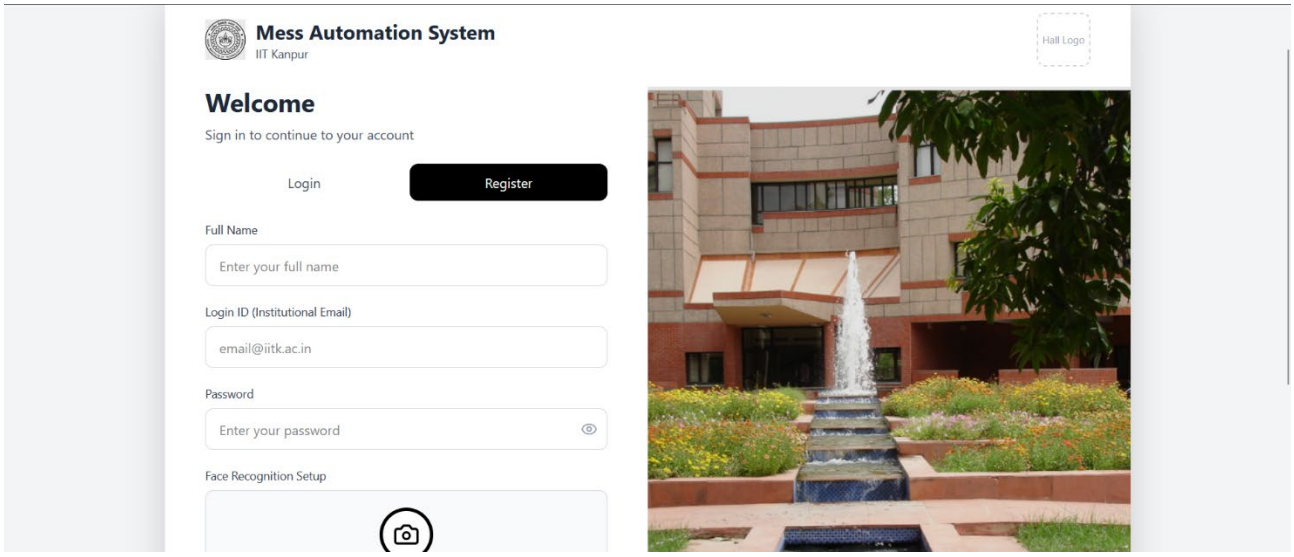


1.2 Human Interface Design

1. Login Page:

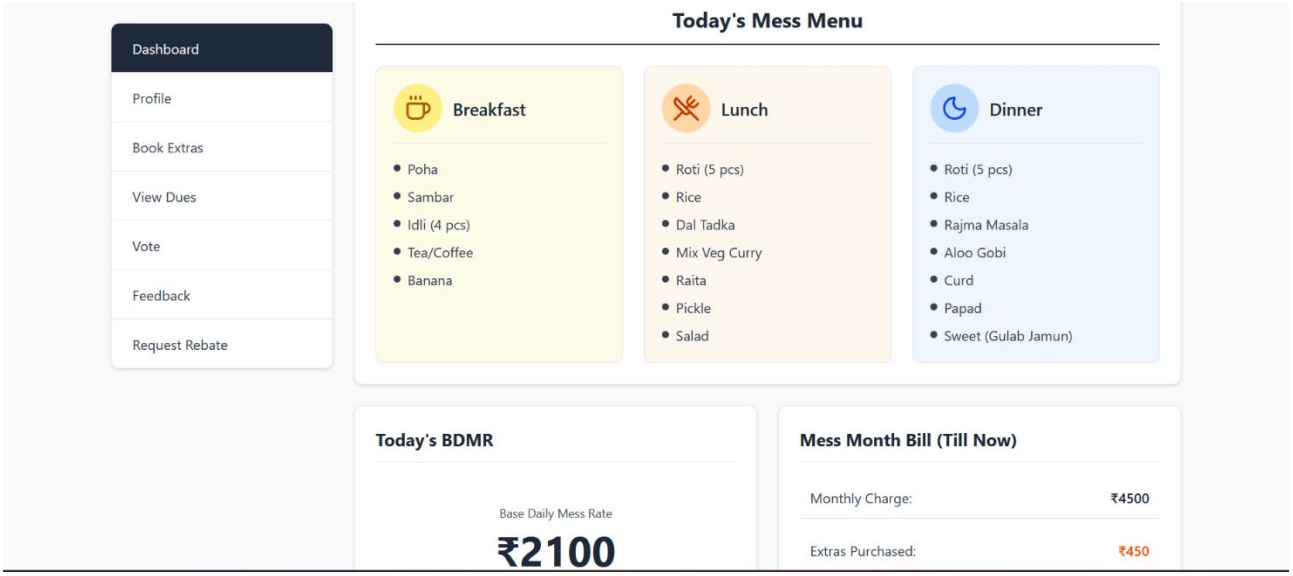


2. Sign Up Page:

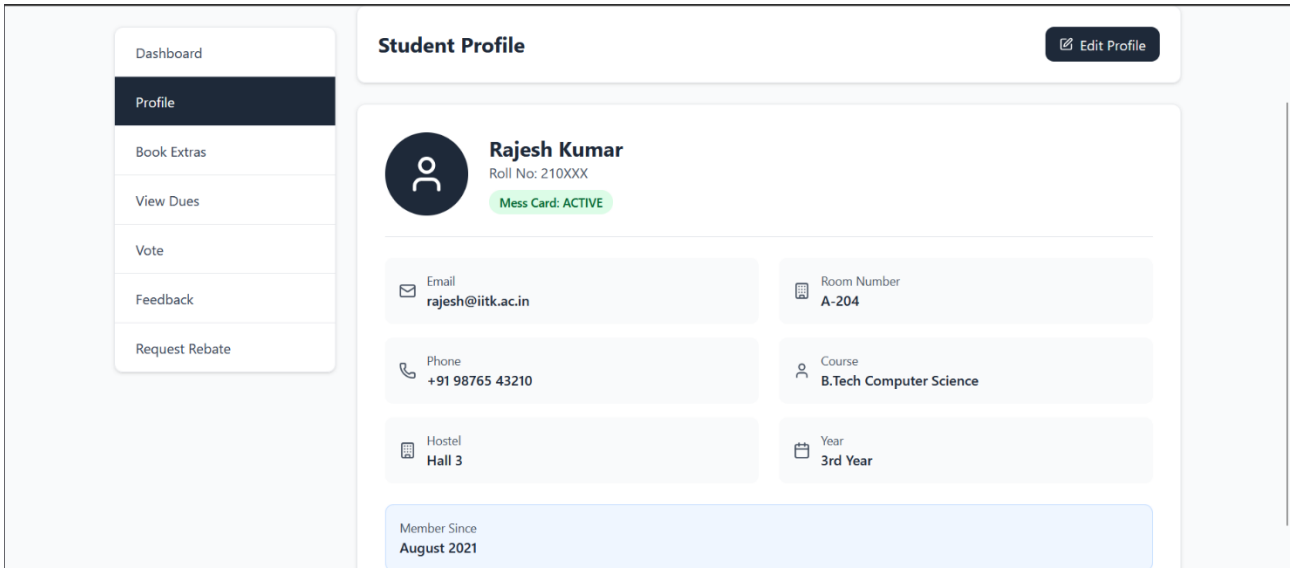


- **Student View:**

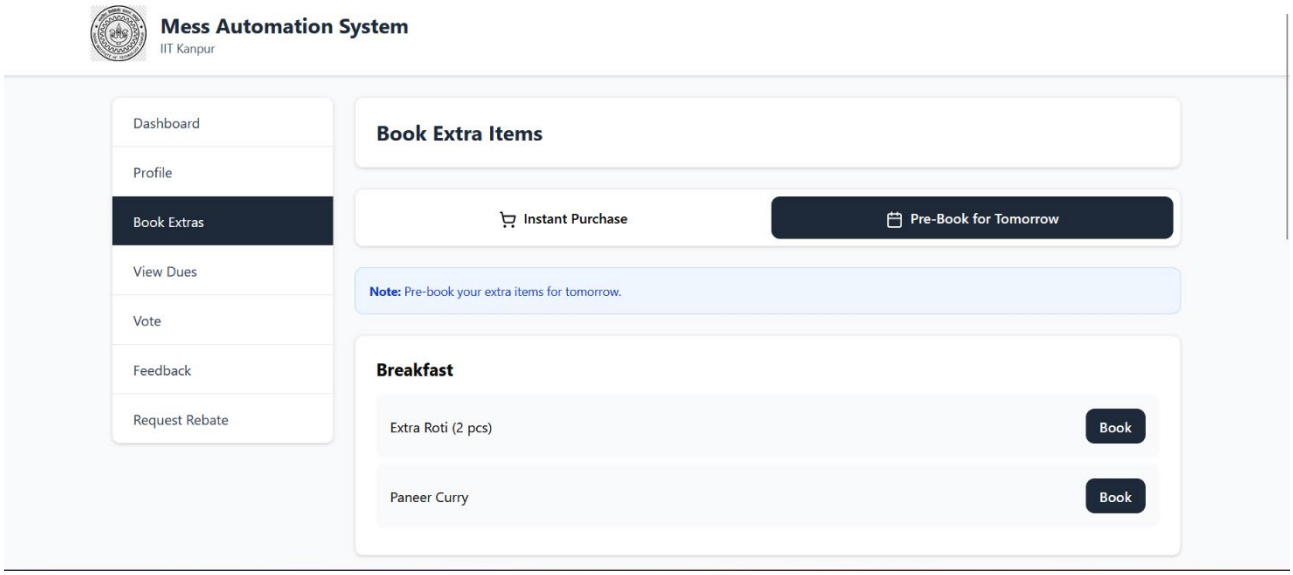
1. Home Page (Dashboard):



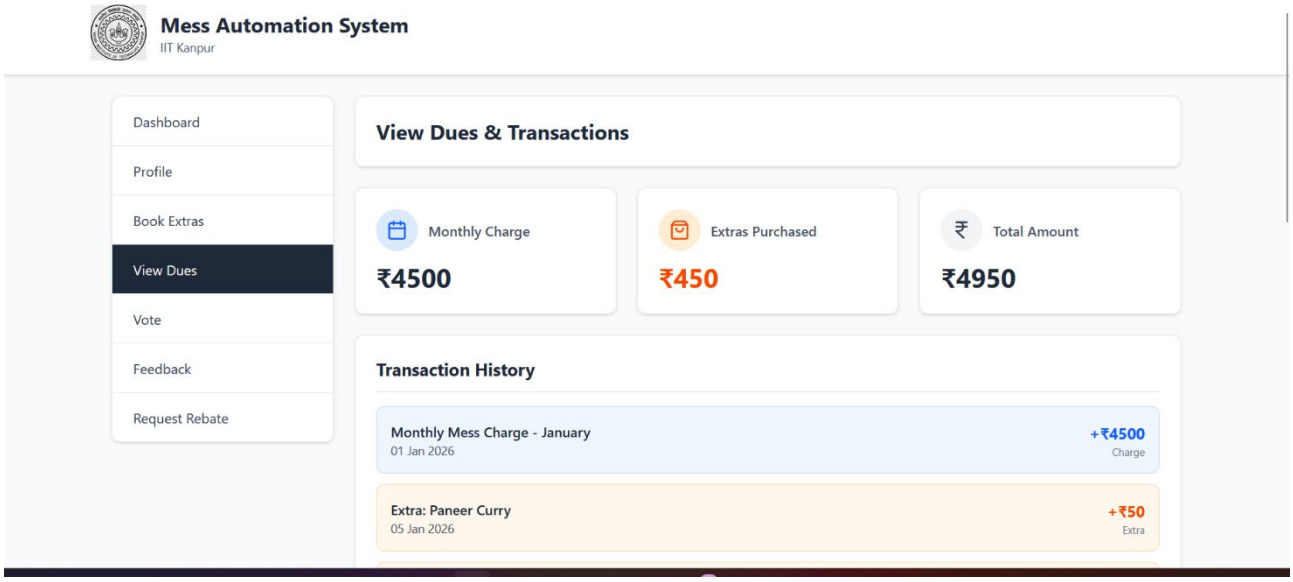
2. Student Profile:



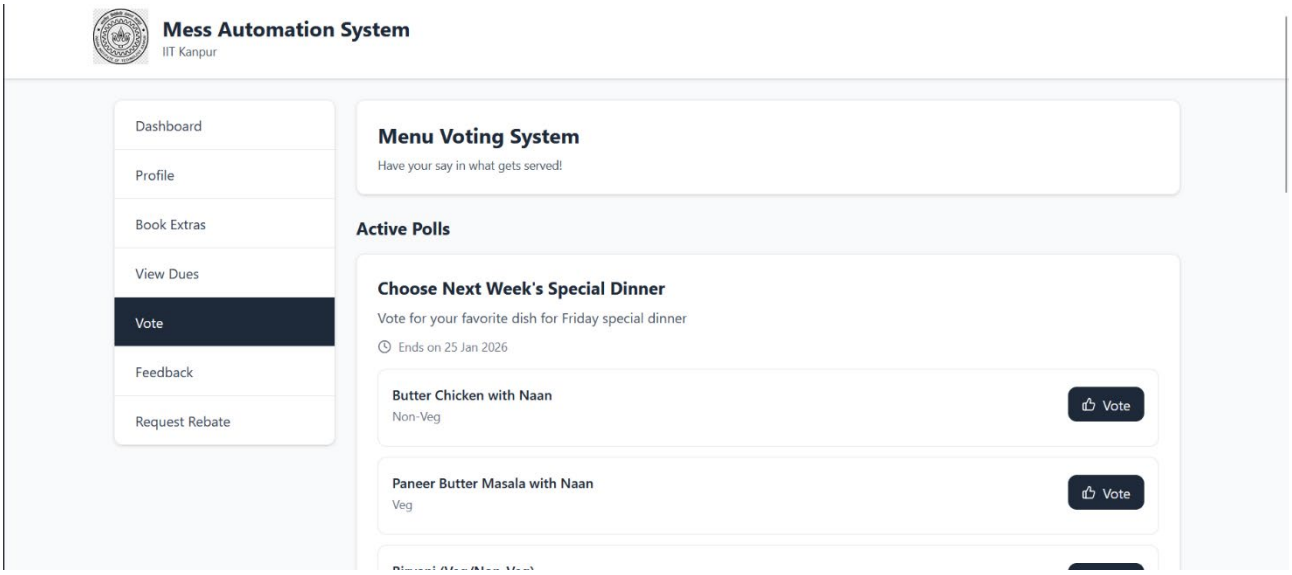
3. Book Extra Item Page:



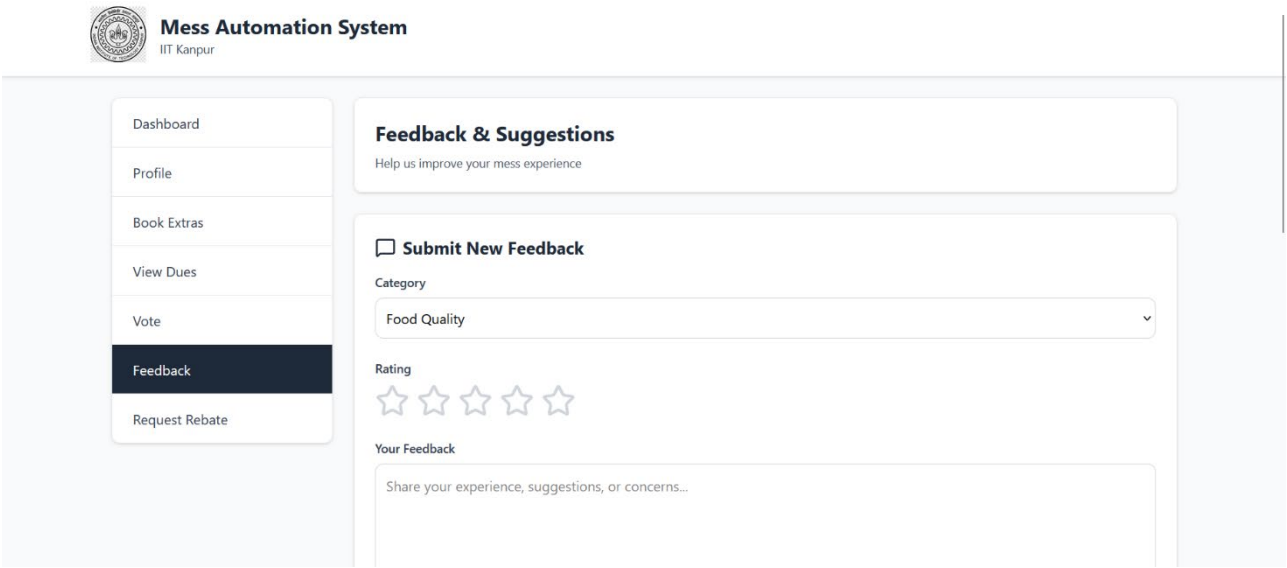
4. View Dues and Transactions Page:



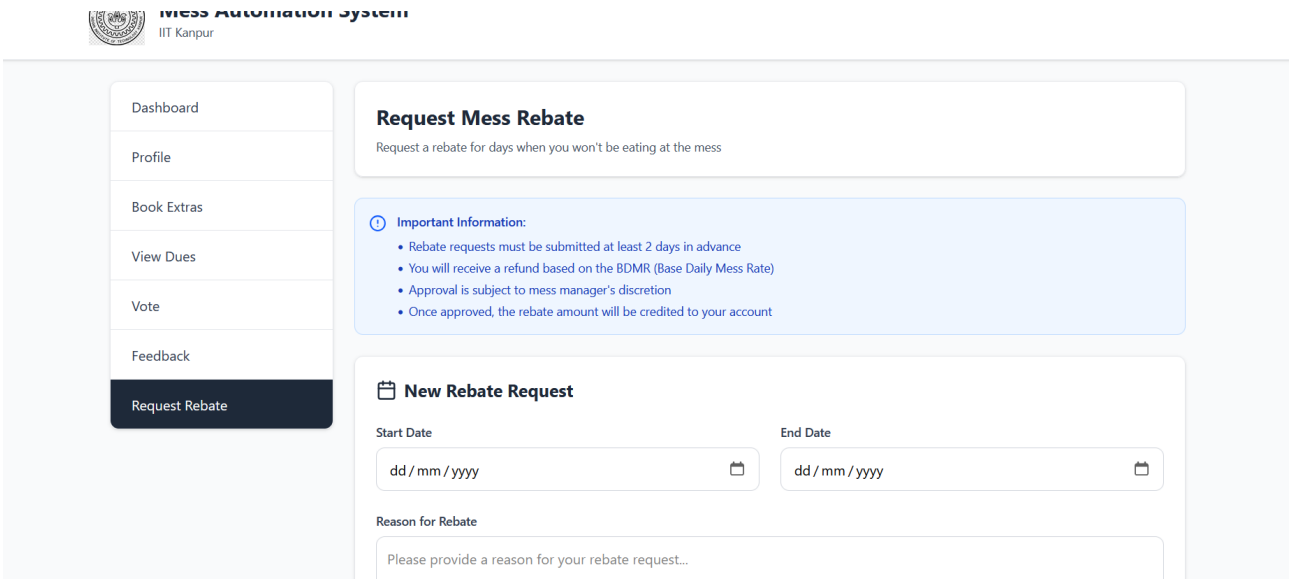
5. Menu Voting System Page:



6. Feedback and suggestions Page:

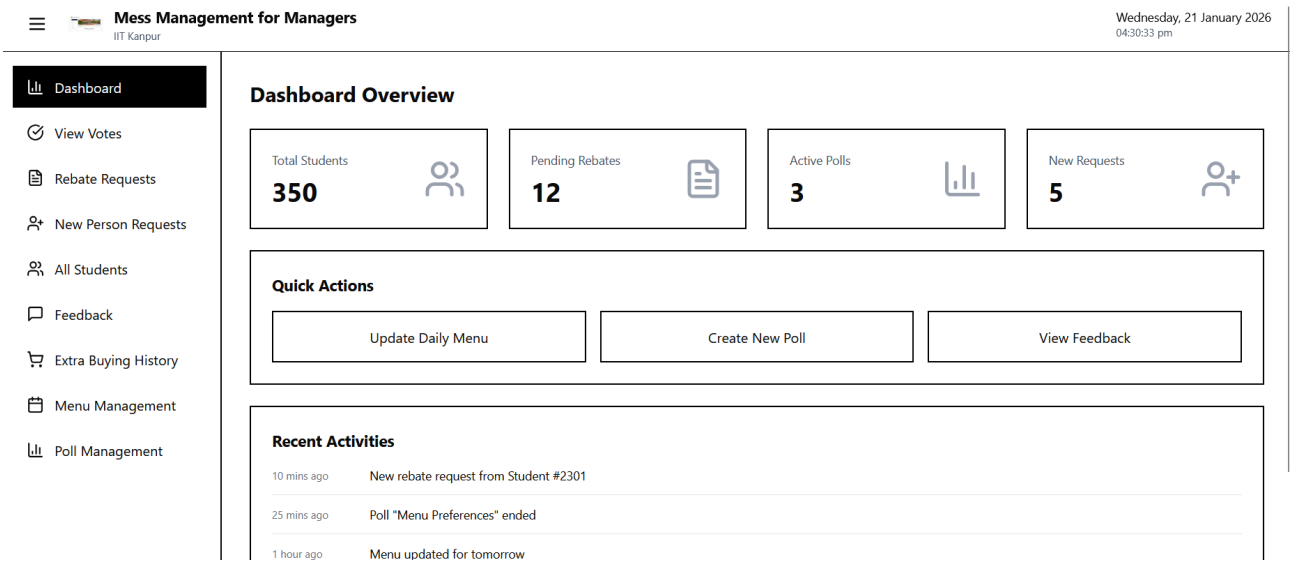


7. Request Mess Rebate Page:

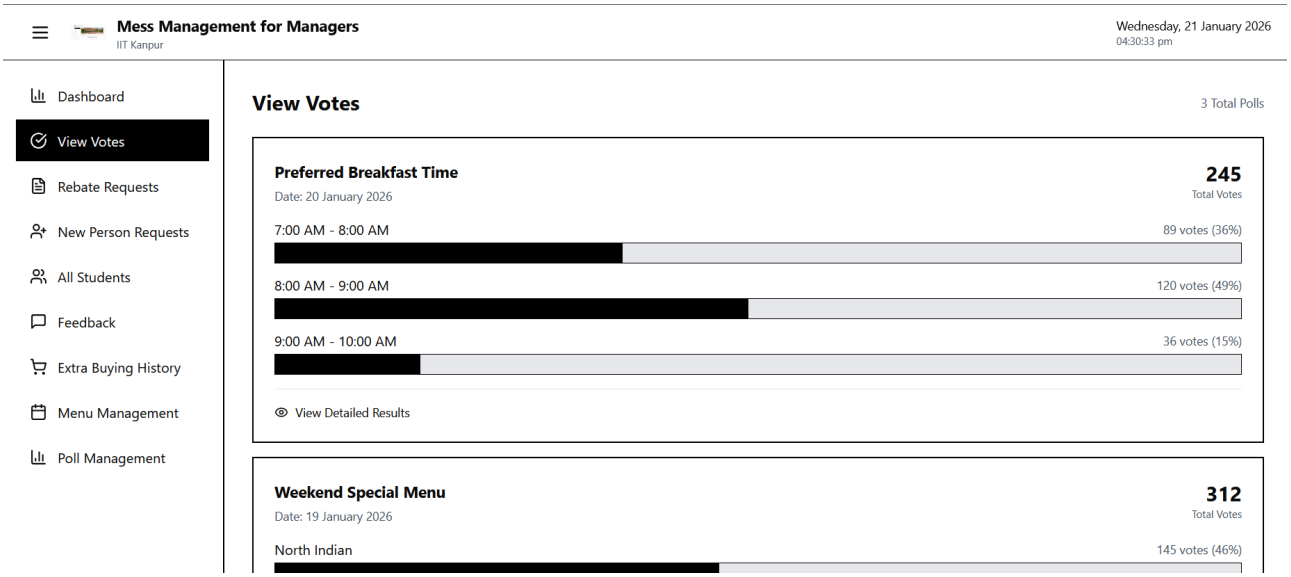


- **Manager View:**

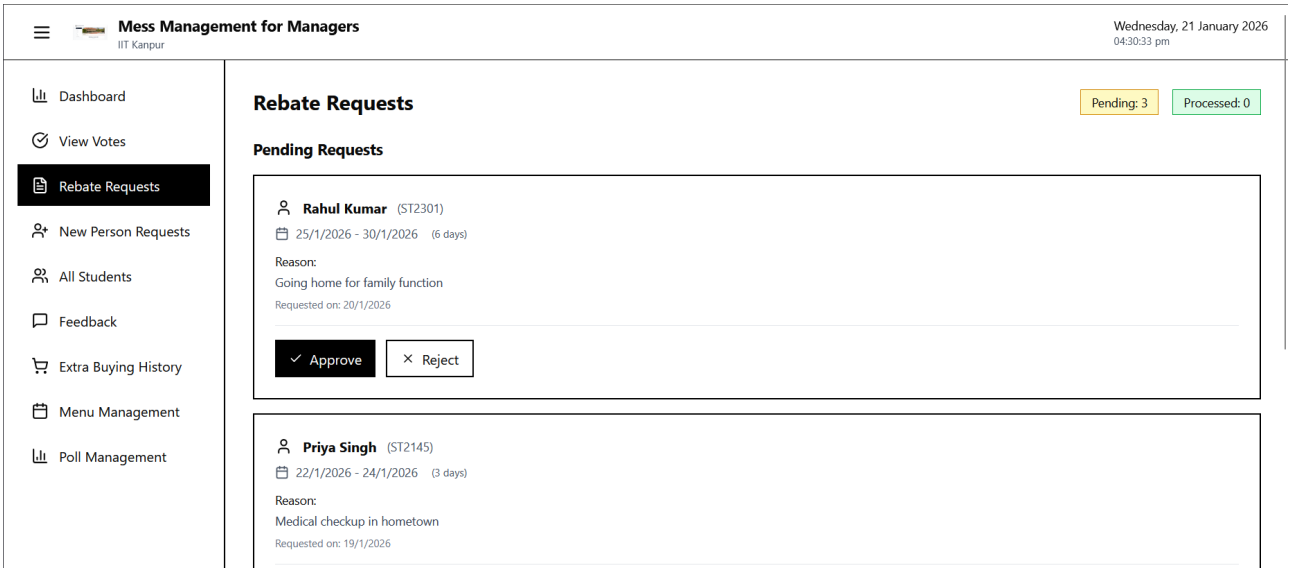
1. Manager Dashboard:



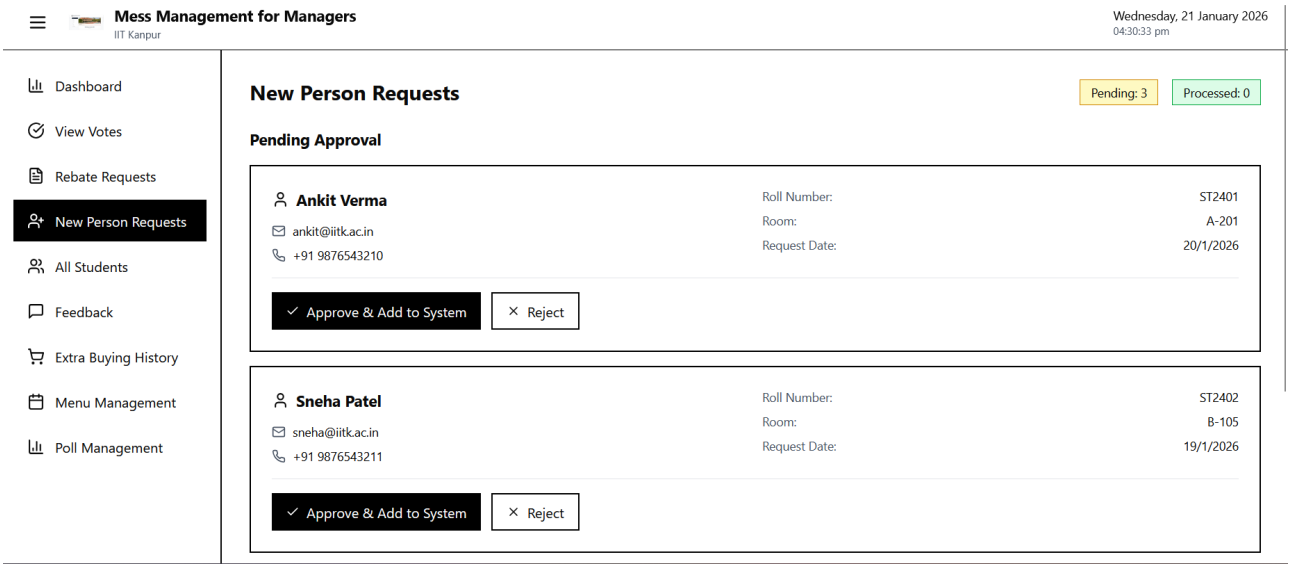
2. View Votes – Brief Description:



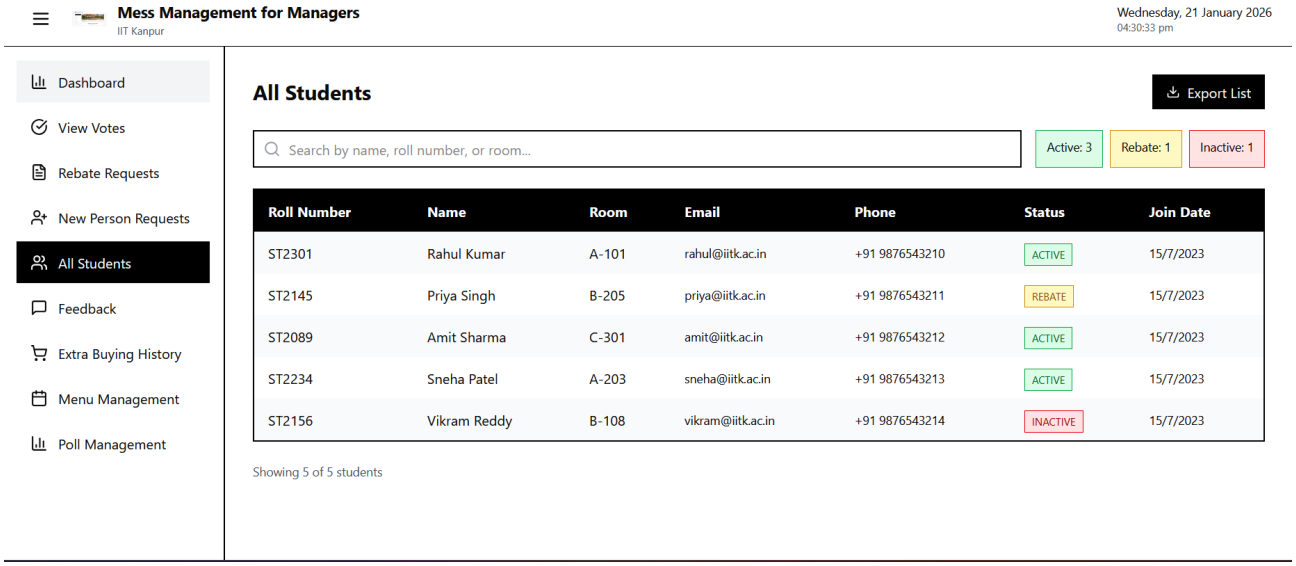
3. Rebate Request:



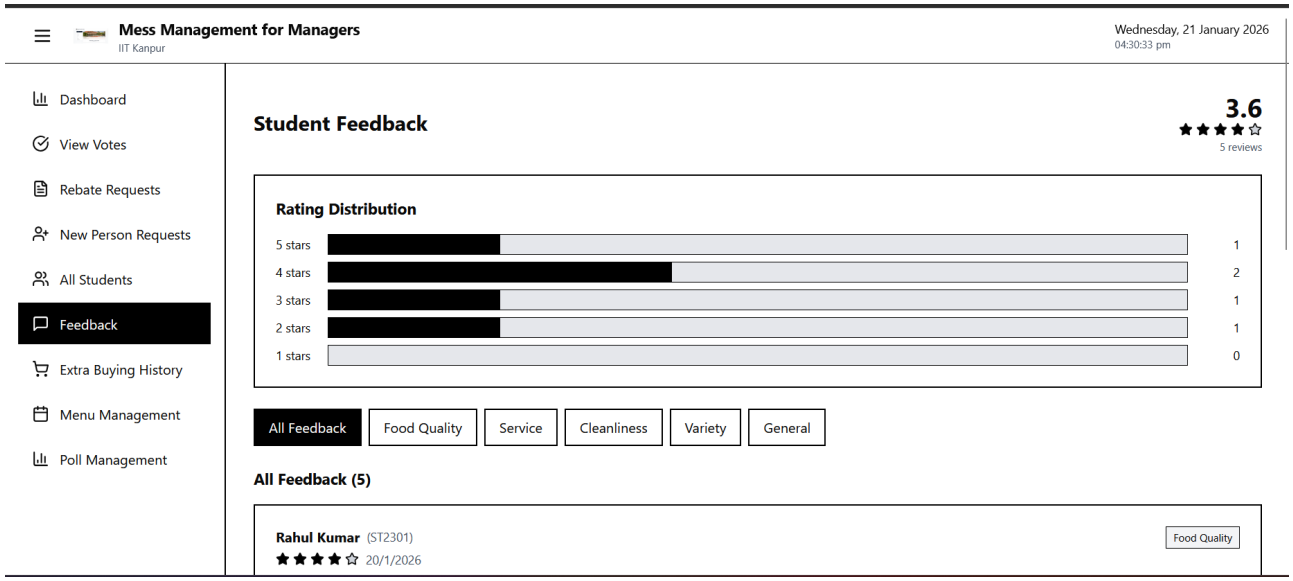
4. New Person Request:



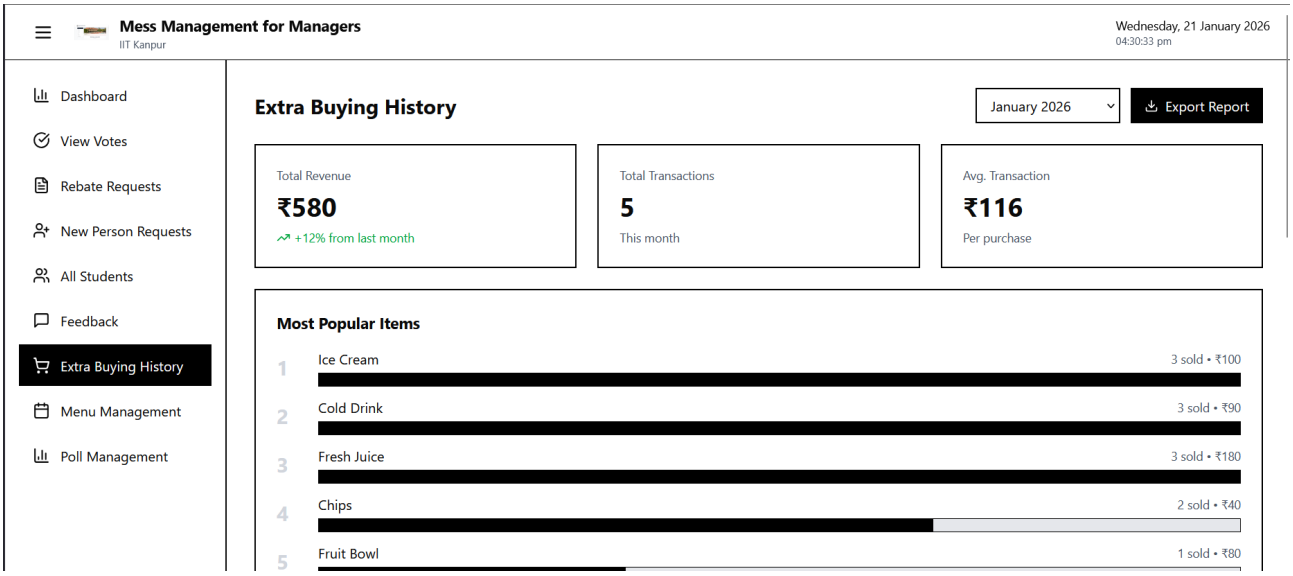
5. All Student – Brief Description:



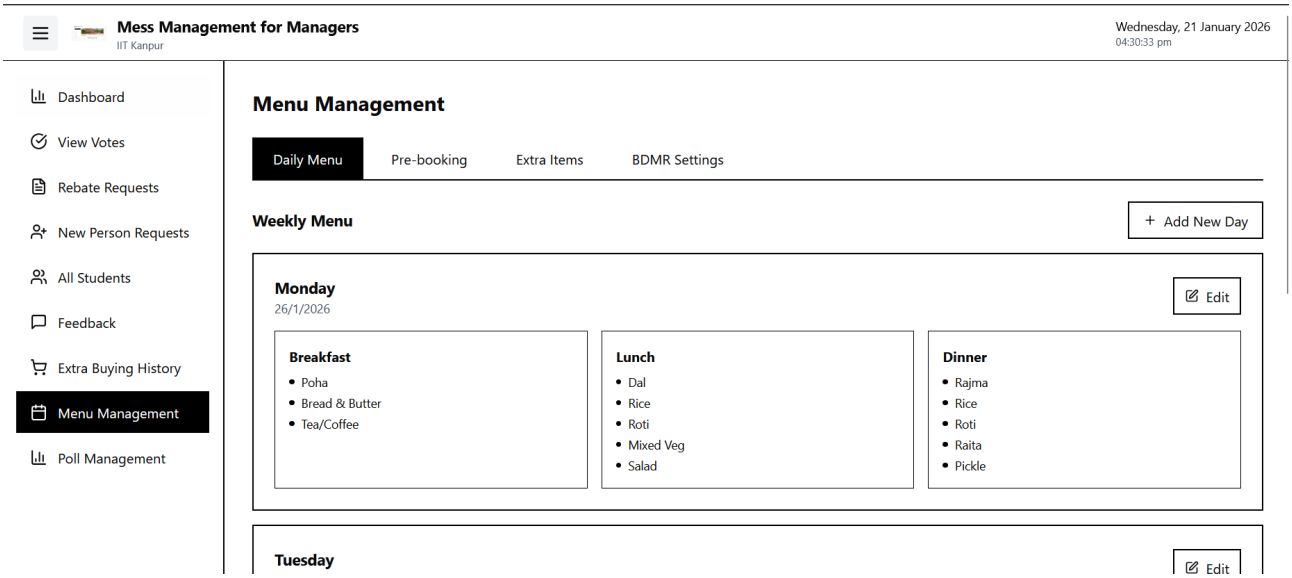
6. Student Feedback:



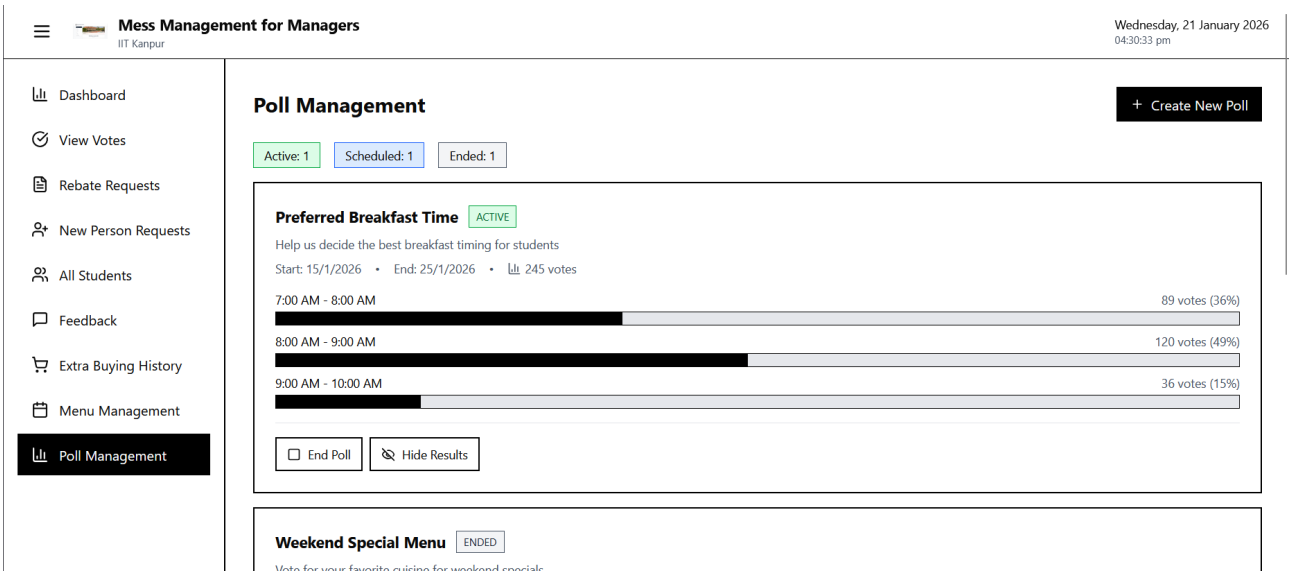
7. Extra Buying History:



8. Menu Management Page:

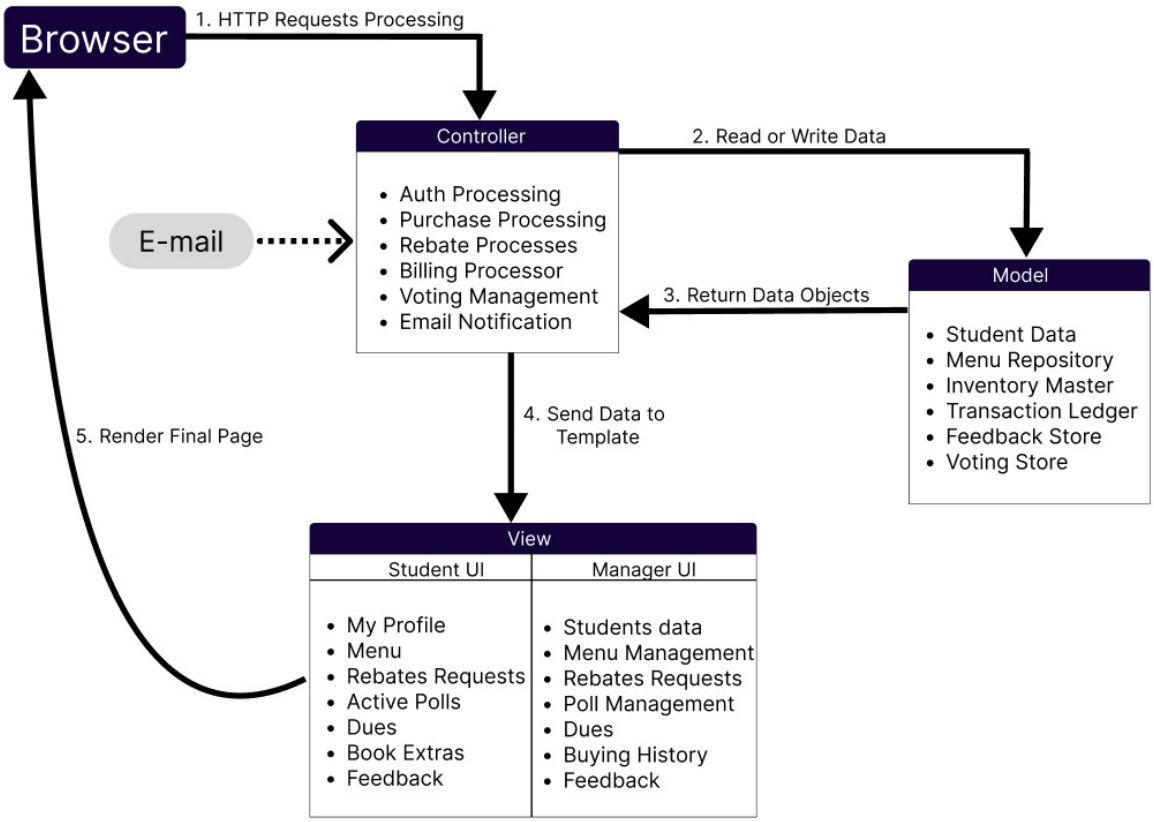


9. Poll Management Page:



2 Architecture Design

2.1 Architecture Design:

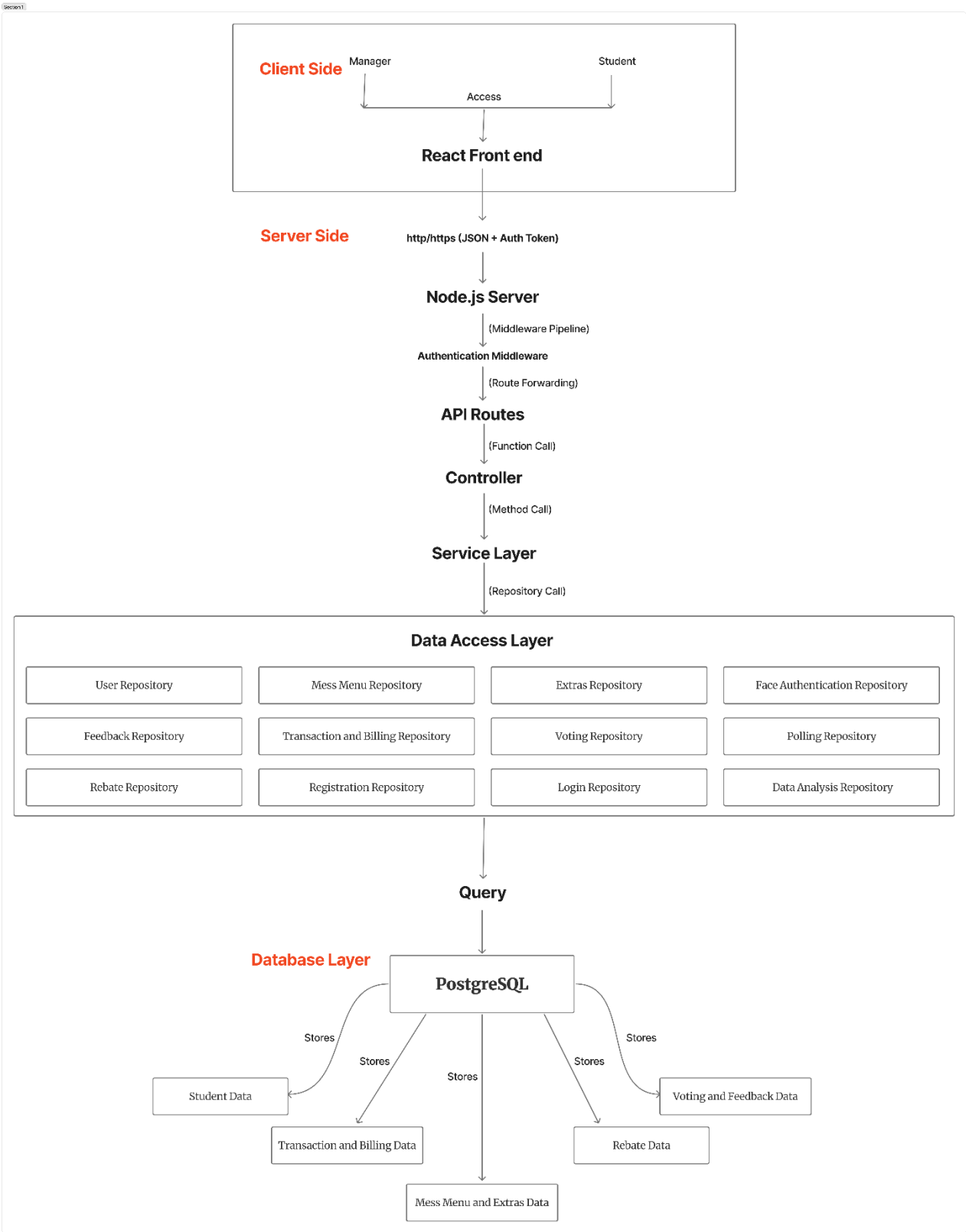


After reviewing the project requirements, we decided to use the **Model-View-Controller (MVC)** design method for the Mess Automation System.

We chose MVC because it is the industry standard for web apps and solves our biggest challenges perfectly:

- **It's the Standard for Web Apps:** MVC is very common for this type of project because it helps keep complex code organized and easy to manage.
- **Separating Data from looks:** The main reason we chose it is that it separates the raw data from what the user sees on the screen.
- **One Database, Different Views:** This allows us to maintain one uniform database that is safe and secure. At the same time, we can easily show different screens to different people based on who they are (like showing a voting screen to a Student but an inventory screen to a manager).

2.2 Tech Stack Architecture



The system follows a **client–server architecture**.

On the **client side**, both **students and managers** access the system through a **React frontend**, which provides role-based dashboards. The frontend communicates with the backend using **HTTP/HTTPS requests with JSON data and authentication tokens**.
On the **server side**, a **Node.js backend** handles all requests. Each request passes through an **authentication middleware** (including optional face authentication using open-source tools like OpenCV), then moves through **API routes, controllers, and the service layer**, where business logic is applied. Database operations are handled by a **Data Access Layer (DAL)** using repositories, which interact with a **PostgreSQL database** to store and retrieve all mess-related data such as users, menus, transactions, voting, feedback, and rebates.

This layered approach keeps the system modular, secure, and easy to maintain.

Why This Architecture Meets the Non-Functional Requirements

1. **Scalability**

The client and server are clearly separated, which allows the backend (Node.js server) to be scaled independently as the number of users increases. The database layer is centralized and structured, making it suitable for handling growing data such as transactions, votes, and feedback. This design supports future scaling using multiple server instances if required.

2. **Performance**

The system uses **RESTful APIs** and **JSON-based communication**, which keeps data exchange lightweight and fast. Node.js handles requests asynchronously, ensuring good responsiveness even when many students and managers are using the system at the same time.

3. **Security & Privacy**

Security is enforced through **authentication middleware**, role-based access (student vs manager), and secure token-based authentication. Sensitive data is handled only on the server side and stored securely in the database. Optional face authentication adds an extra layer of security for user login.

4. **Interoperability**

Since the frontend communicates with the backend using standard **REST APIs**, the system can run on different devices and browsers without issues. This also makes it easy to integrate third-party services in the future if needed.

5. **Reliability**

The clear separation of layers (controller, service, and data access) makes the system easier to test, debug, and maintain. Errors can be isolated quickly, and updates can be made without affecting the entire system, improving overall reliability.

3 Object Oriented Design

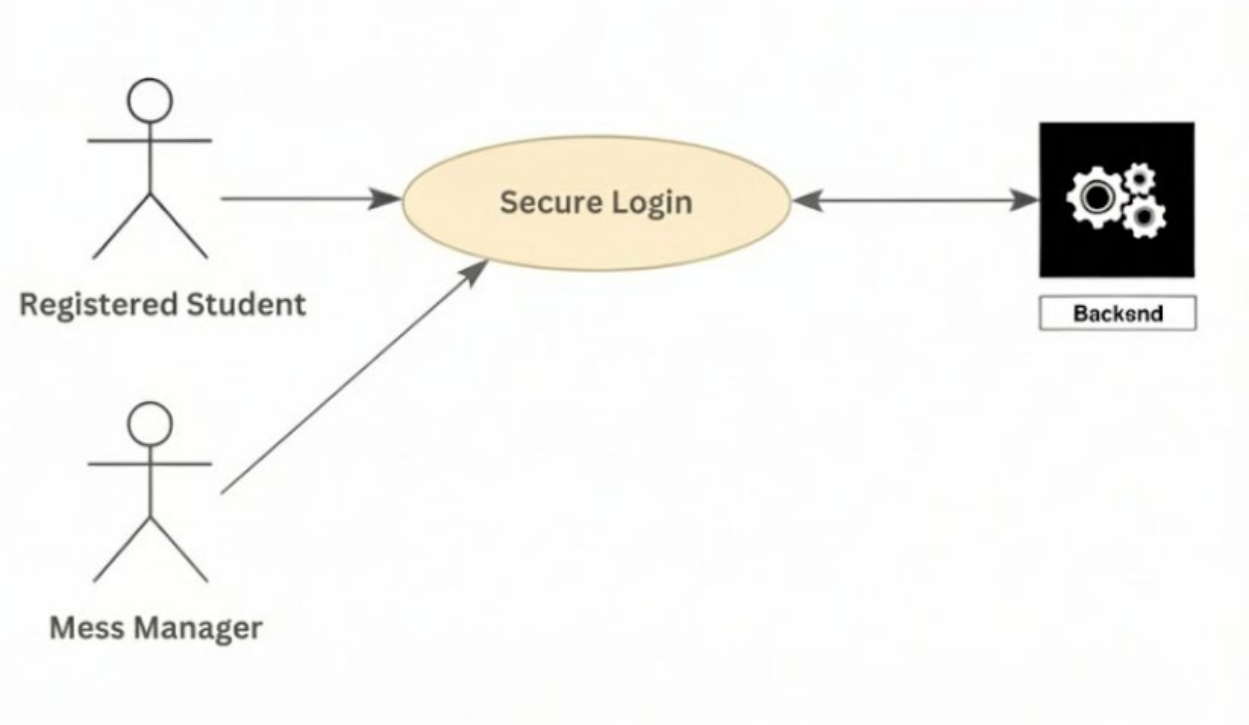
3.1 Use Case Diagrams

3.1.1 Student Registration & Face Enrolment.

Purpose: To register new students and enrol their biometric data for mess entry.

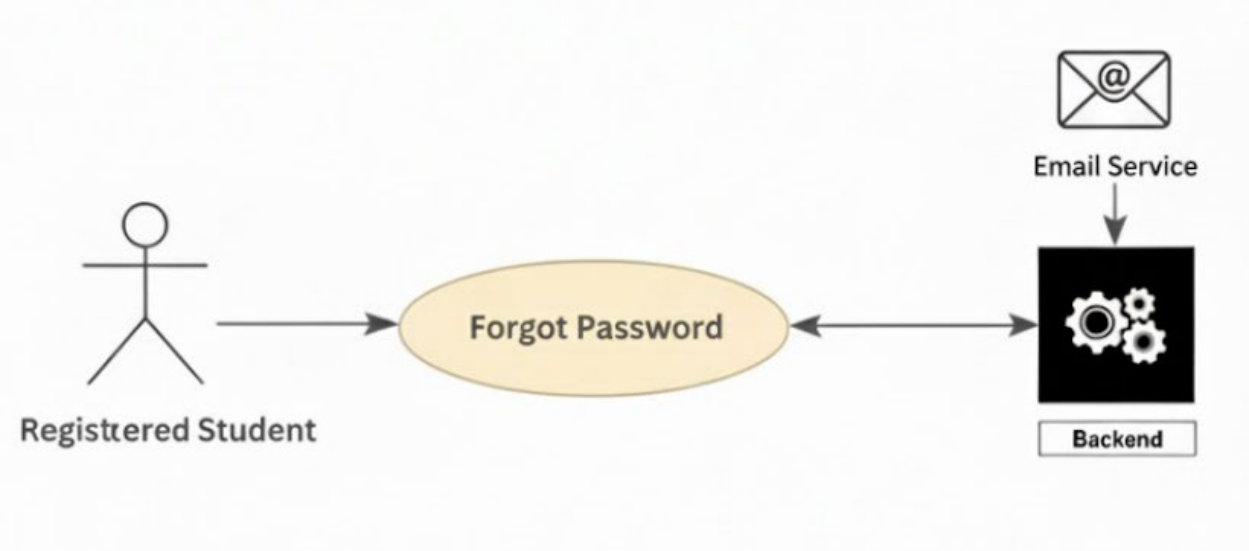
3.1.2 Secure Login (Password/Face ID)

Purpose: To authenticate users (Students/Admin) into the system using multi-modal methods.



3.1.3 Forgot Password

Purpose: To allow students to securely reset their login credentials without administrative help.

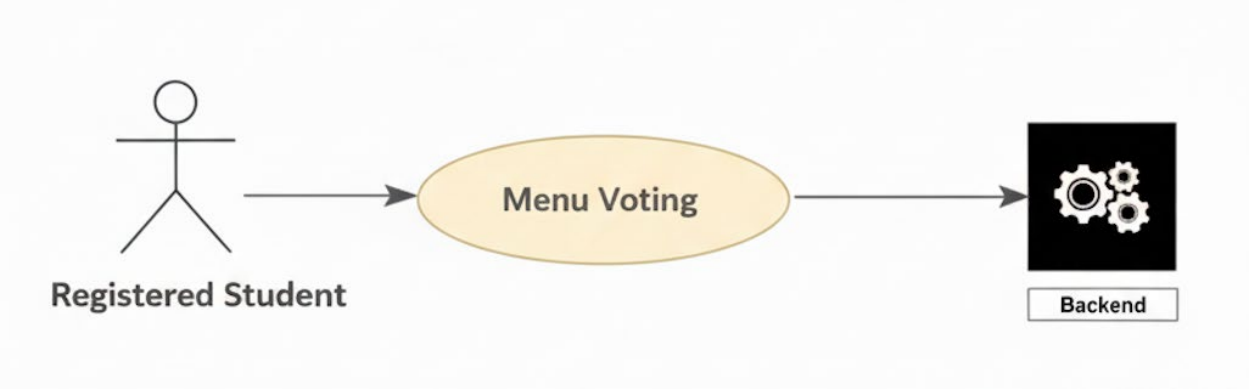


3.1.4 Student Profile Management

Purpose: To allow students to view their personal, academic, and membership details and update editable information.

3.1.5 Menu Voting

Purpose : To allow students to participate in deciding the menu via a democratic voting system.



3.1.6 Extra Food Purchase

Purpose: To allow students to buy items not included in the basic menu (e.g., eggs, milk, special dishes).

3.1.7 Rebate (Absentee) Request

Purpose: To allow students to mark themselves as absent for specific meals to save food and reduce their monthly bill.

3.1.8 Feedback Submission

Purpose: To provide a channel for students to rate meal quality and for Admins to address grievances

3.1.9 Automated Monthly Billing & Reporting

Purpose: To calculate the final mess bill for each student, accounting for base fees, extras, and rebates

3.1.10 Consumption Analytics Dashboard

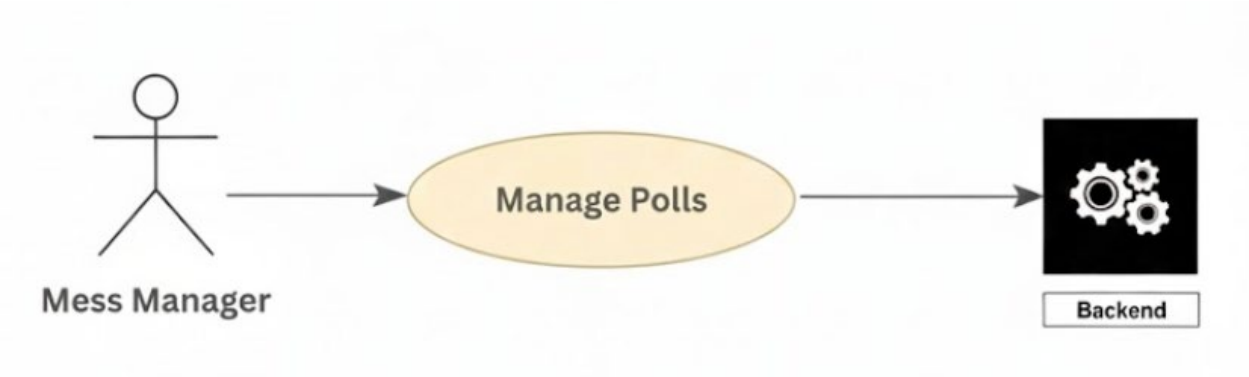
Purpose: To provide the Mess Manager with data-driven insights into food wastage and popularity.

3.1.11 Mess Manager Dashboard Overview

Purpose: To provide a snapshot of mess operations and quick access to administrative modules**.**

3.1.12 View & Manage Polls

Purpose: To conduct student polls, monitor voting trends, and analyse results for menu planning**.**

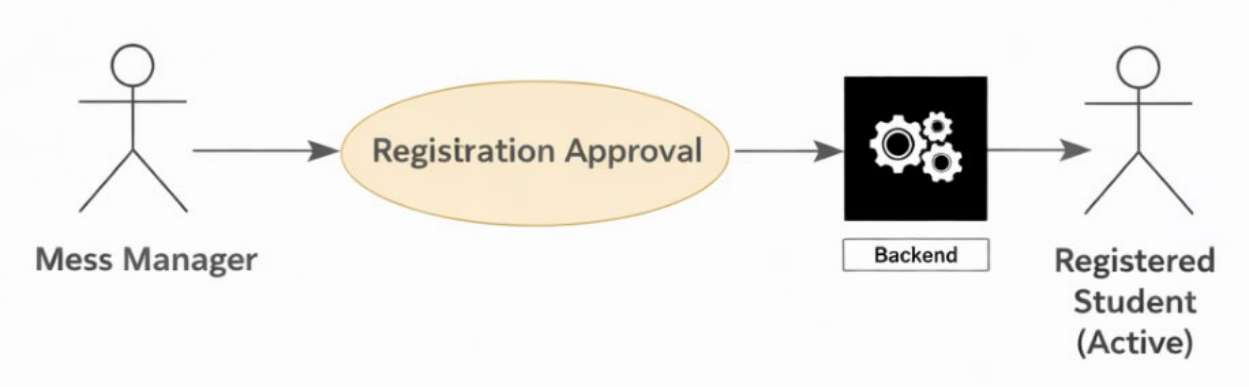


3.1.13 Rebate Request Processing

Purpose: To review and systematically process student rebate applications.

3.1.14 Registration Approval

Purpose: To verify and validate new student entries into the mess system.



3.1.15 Menu Planning & Management

Purpose: To create, edit, and organise weekly mess menus.

3.1.16 Student Management & Search

Purpose: To track and manage the database of all registered students.

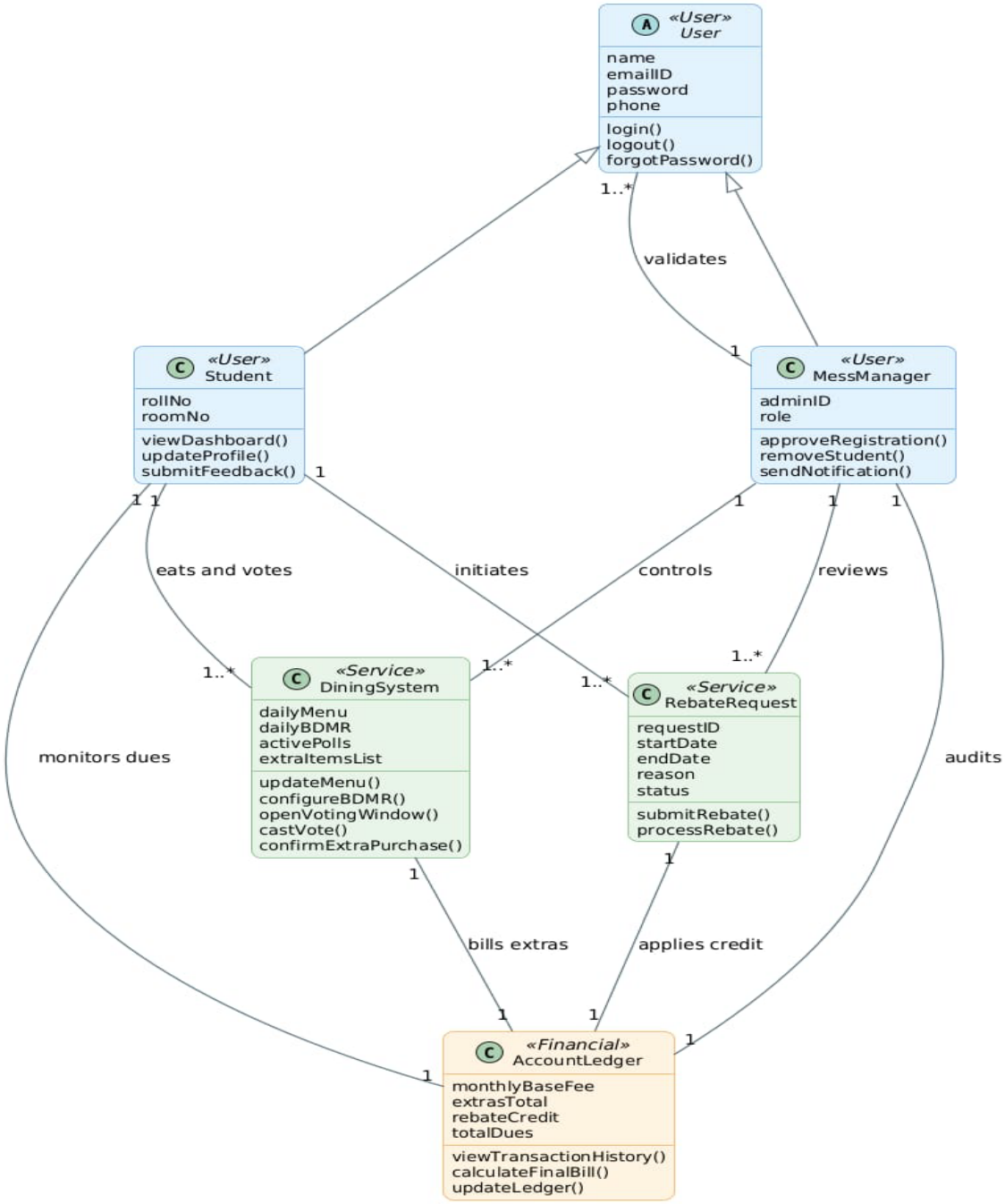
3.1.17 Feedback & Satisfaction Analysis

Purpose: To monitor student satisfaction and identify mess service improvements.

3.1.18 Extra Buying

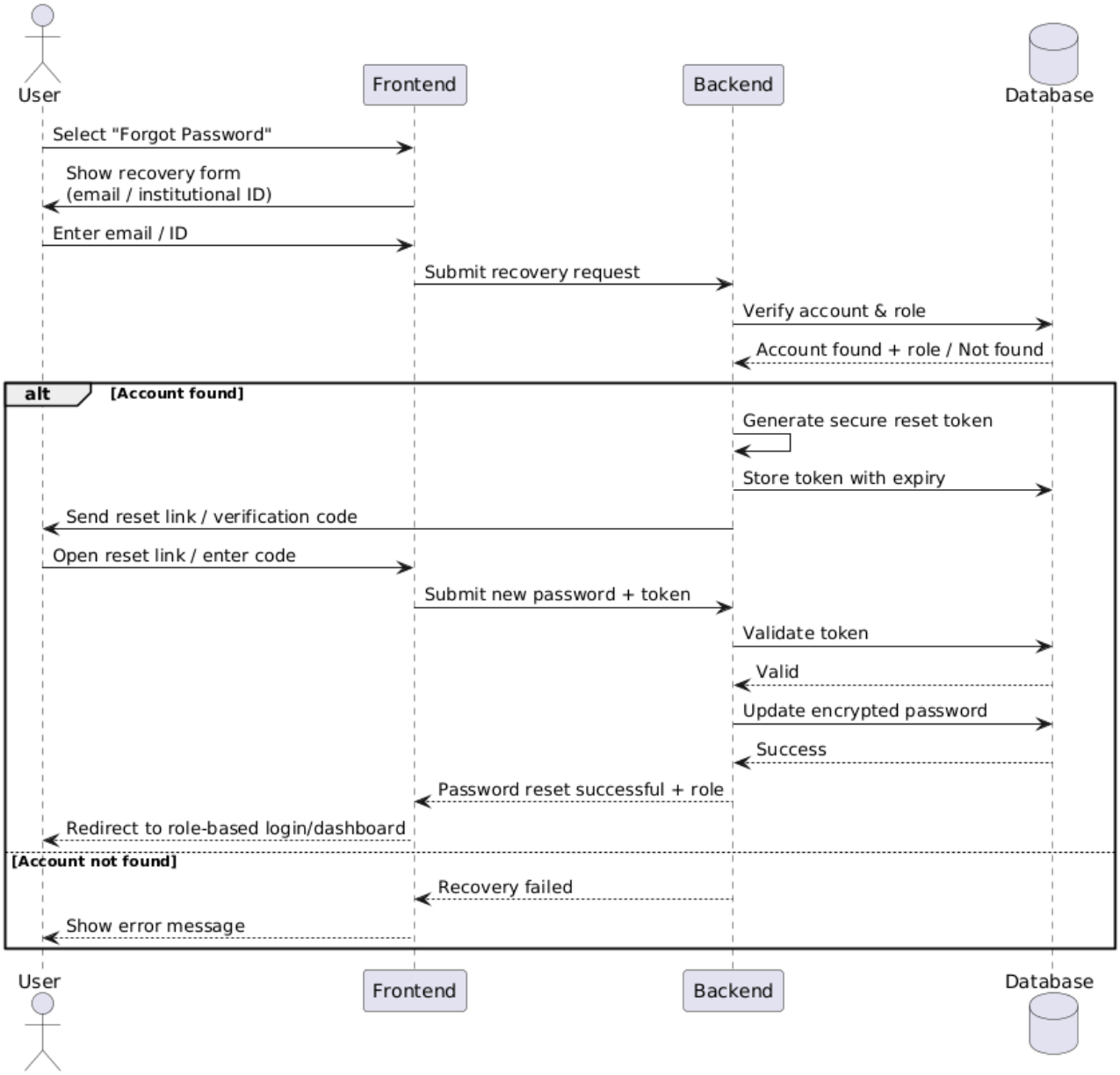
Purpose: To analyse sales trends and monitor revenue from extra food purchases.

3.2 Class Diagrams

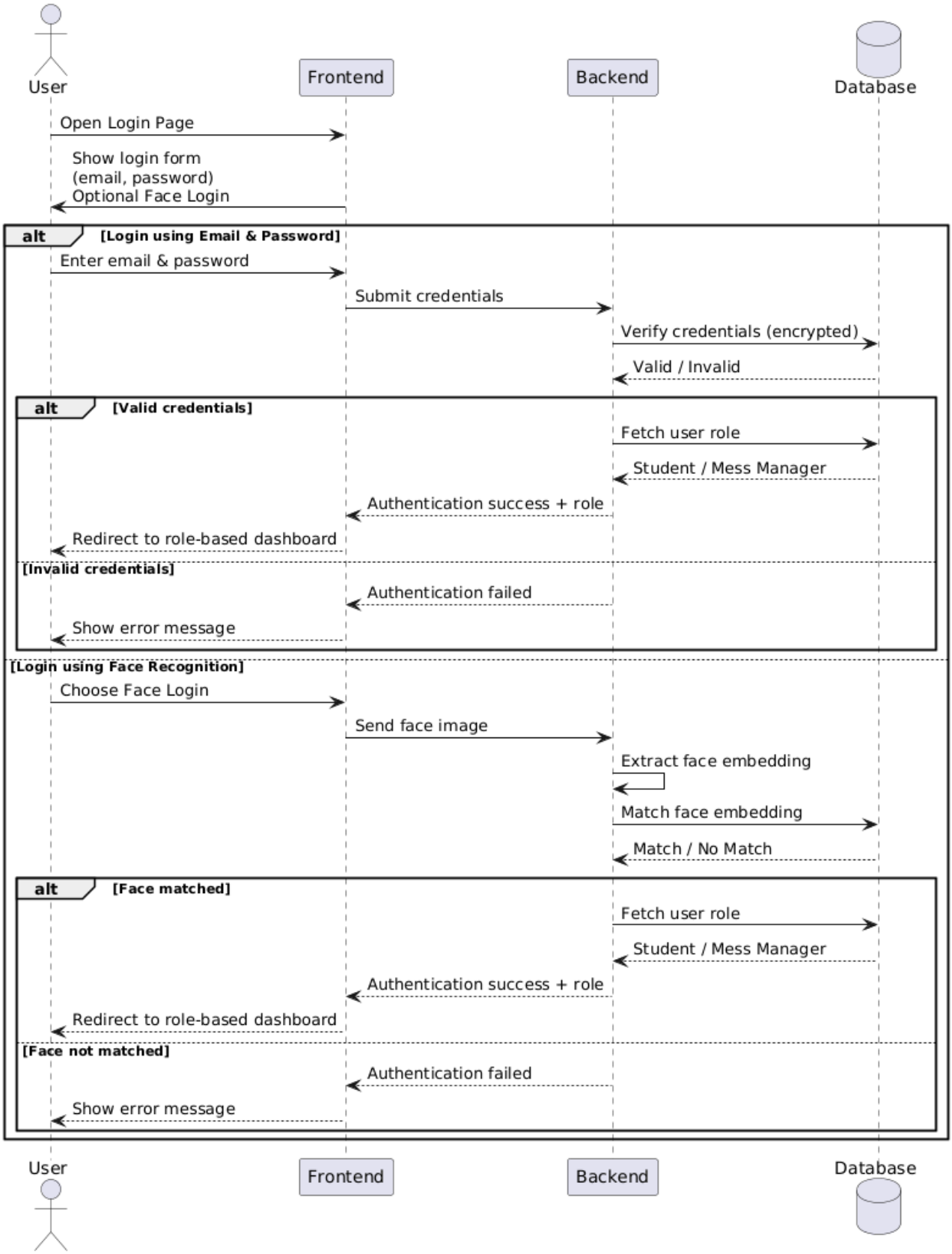


3.3 Sequence Diagrams

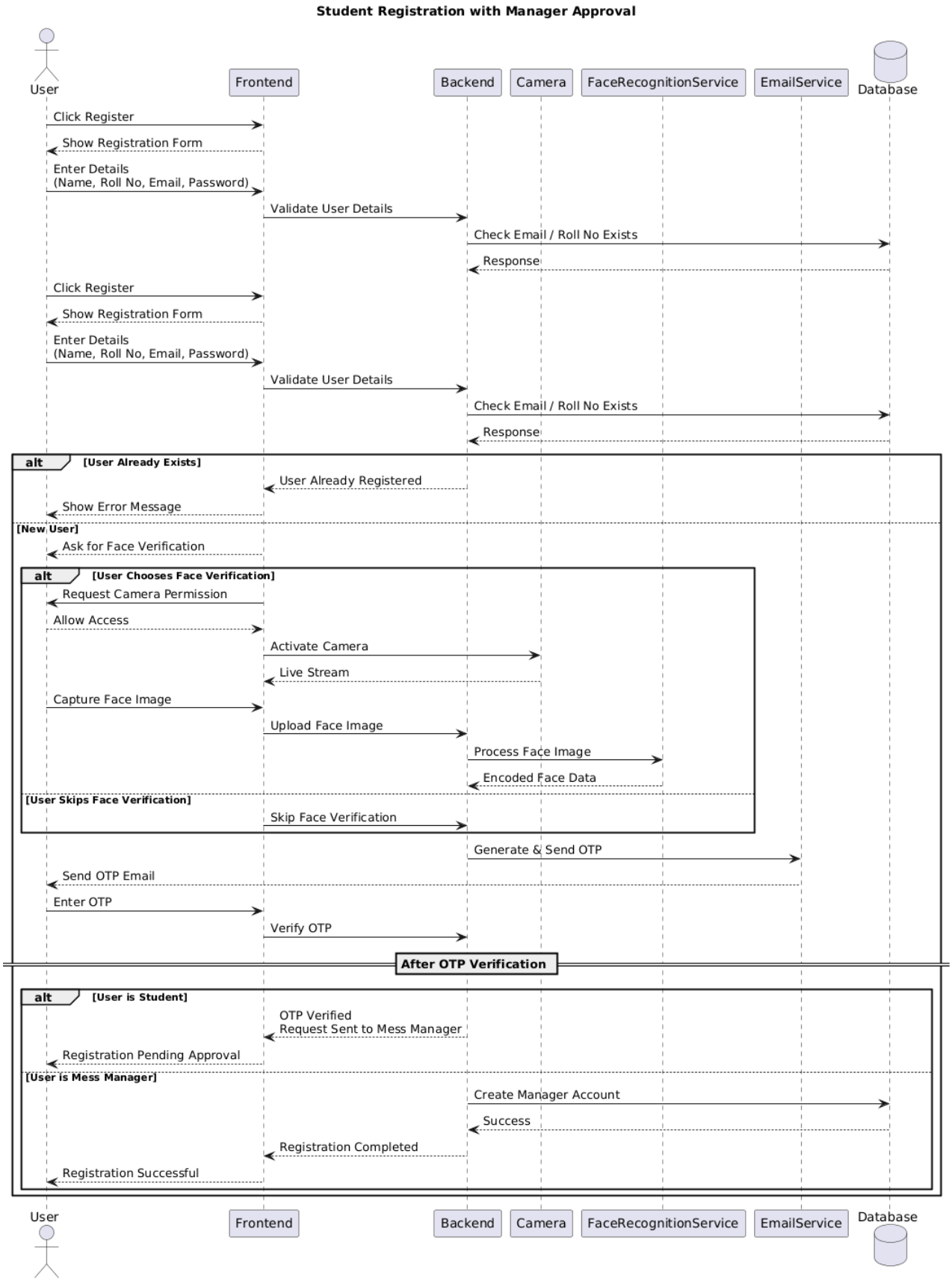
3.3.1 Forgot Password:



3.3.2 Login Page:

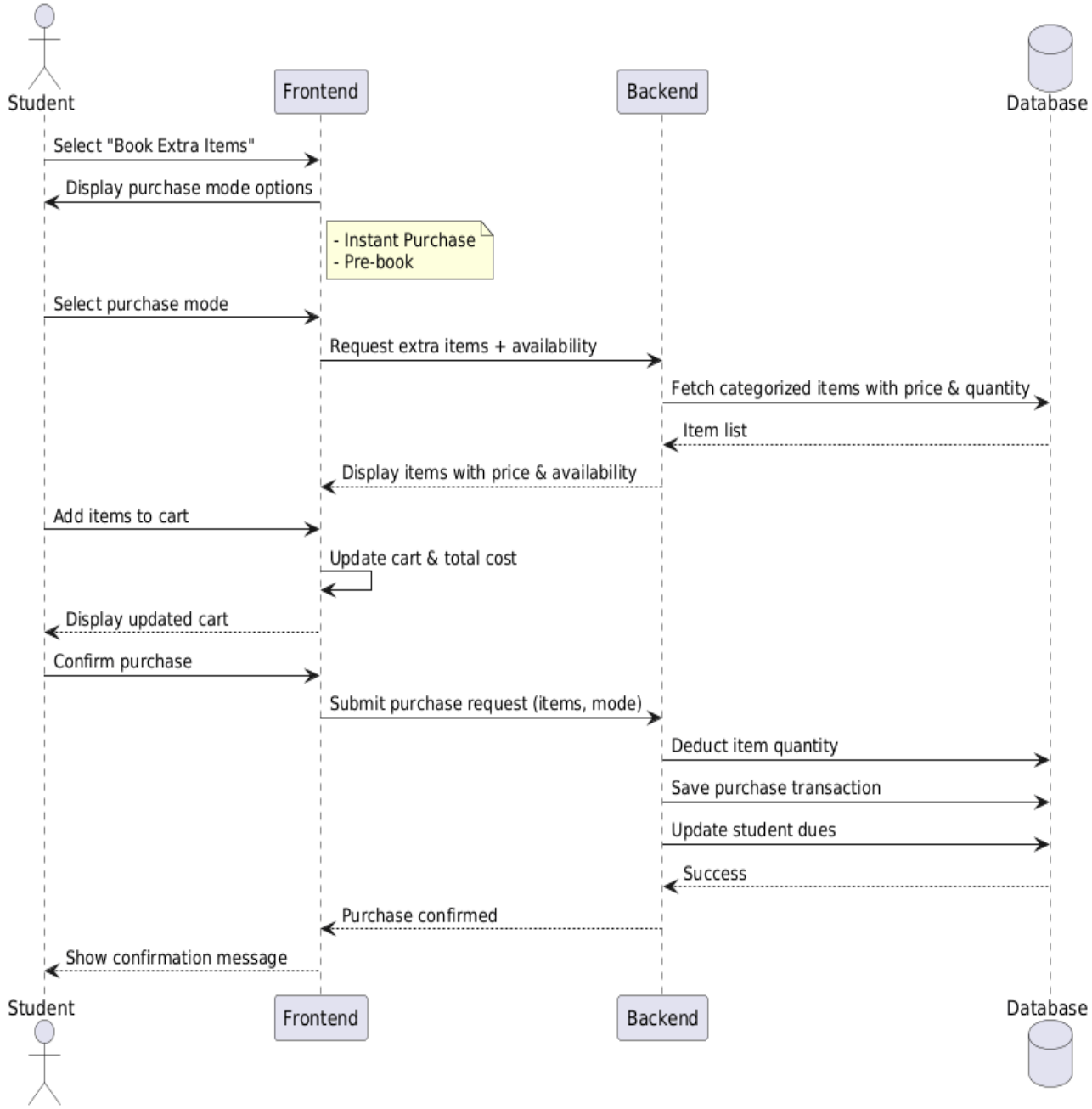


3.3.3 New Registration:

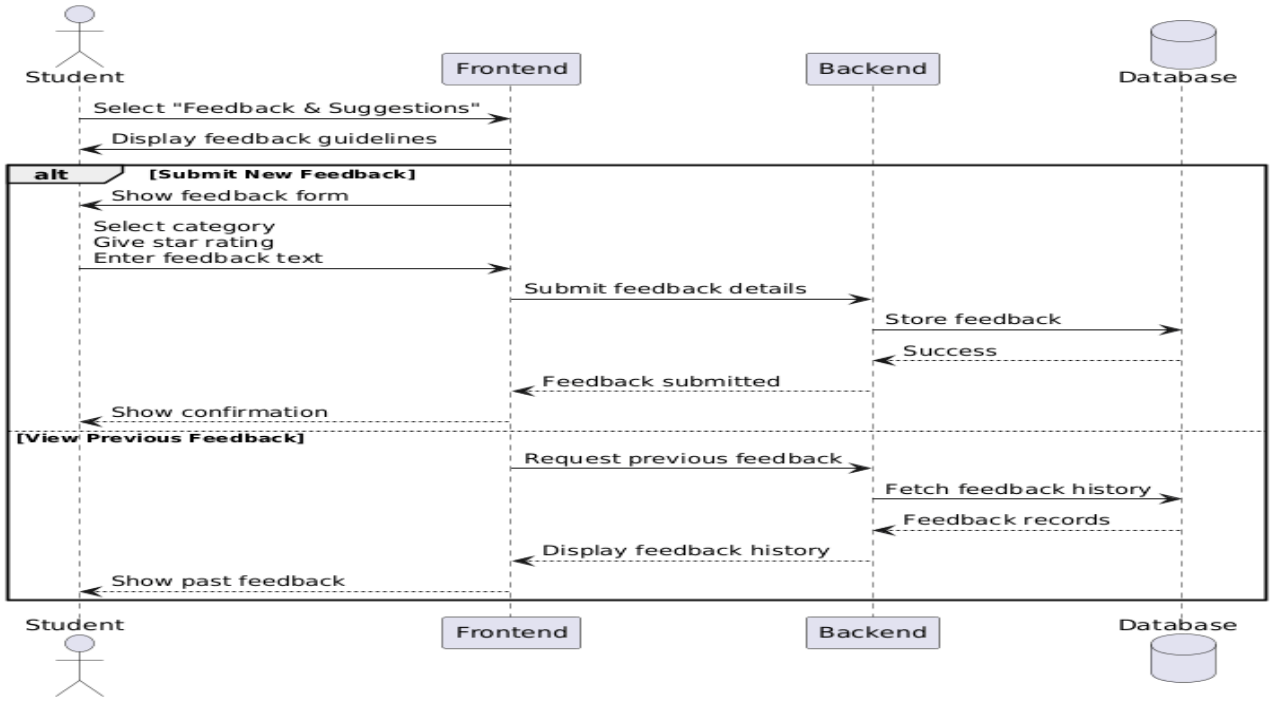


- **Student Sequence Diagram**

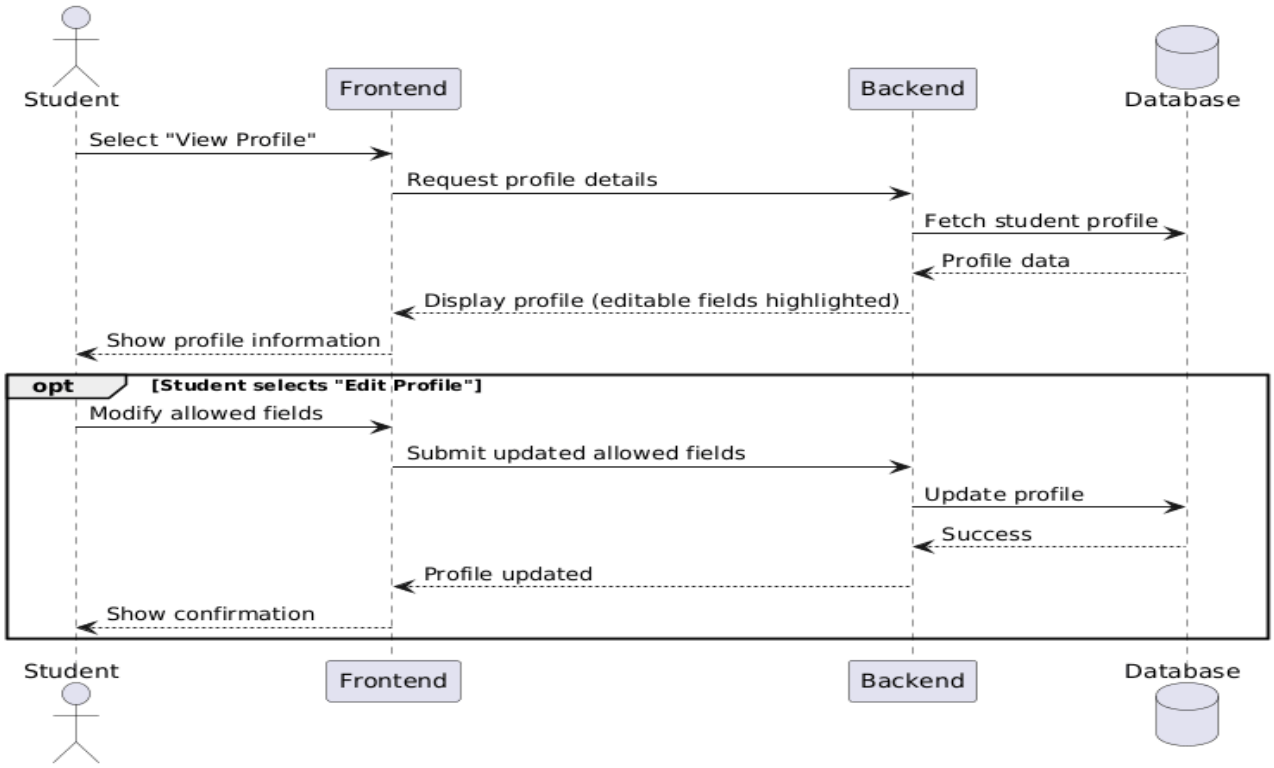
3.3.4 Extras:



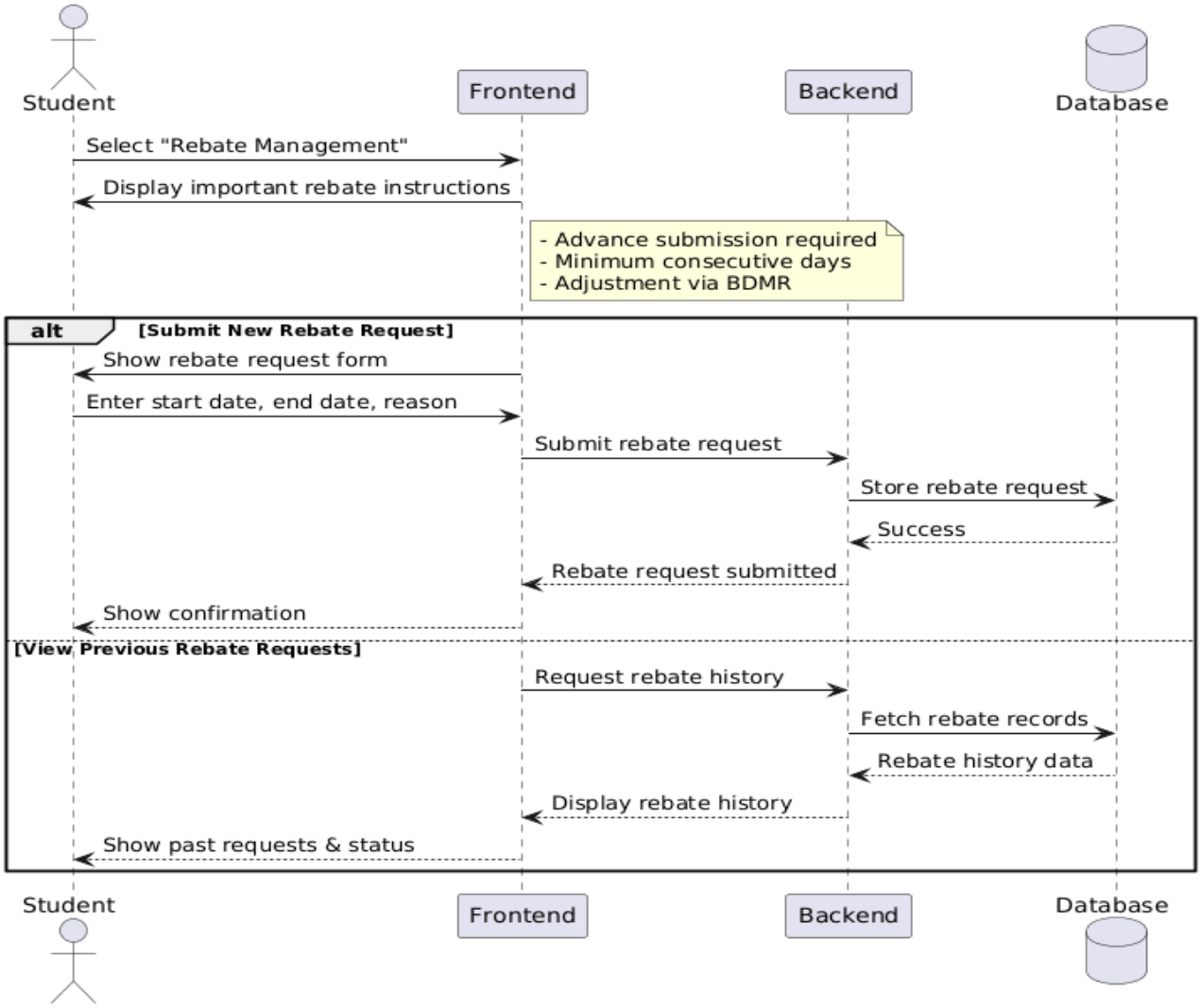
3.3.5 Feedback:



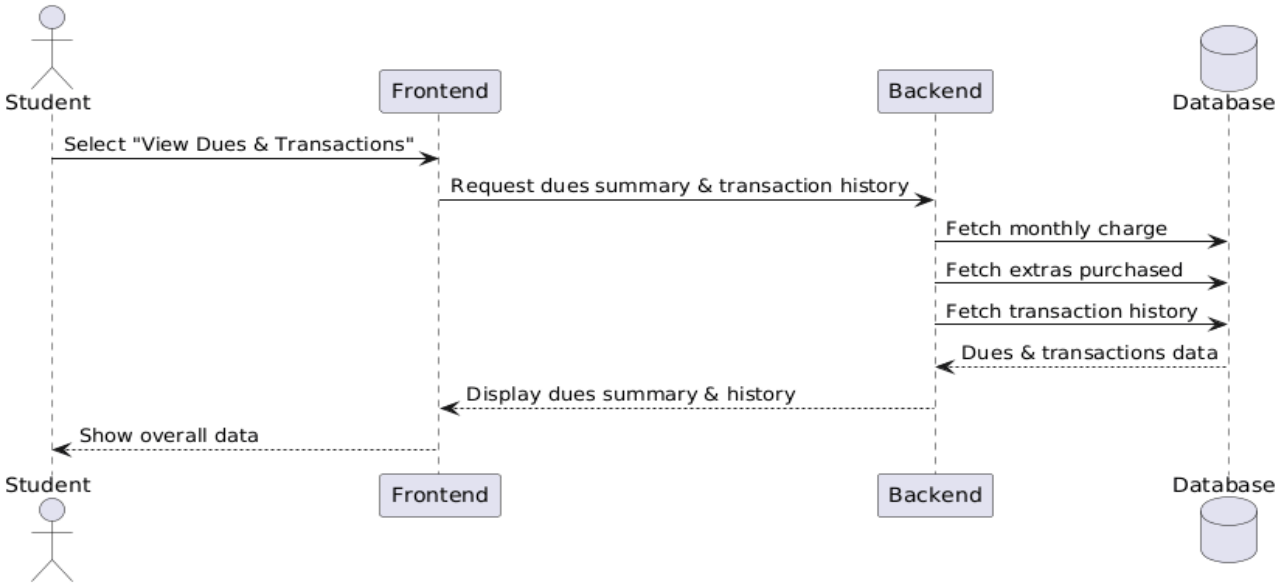
3.3.6 Profile:



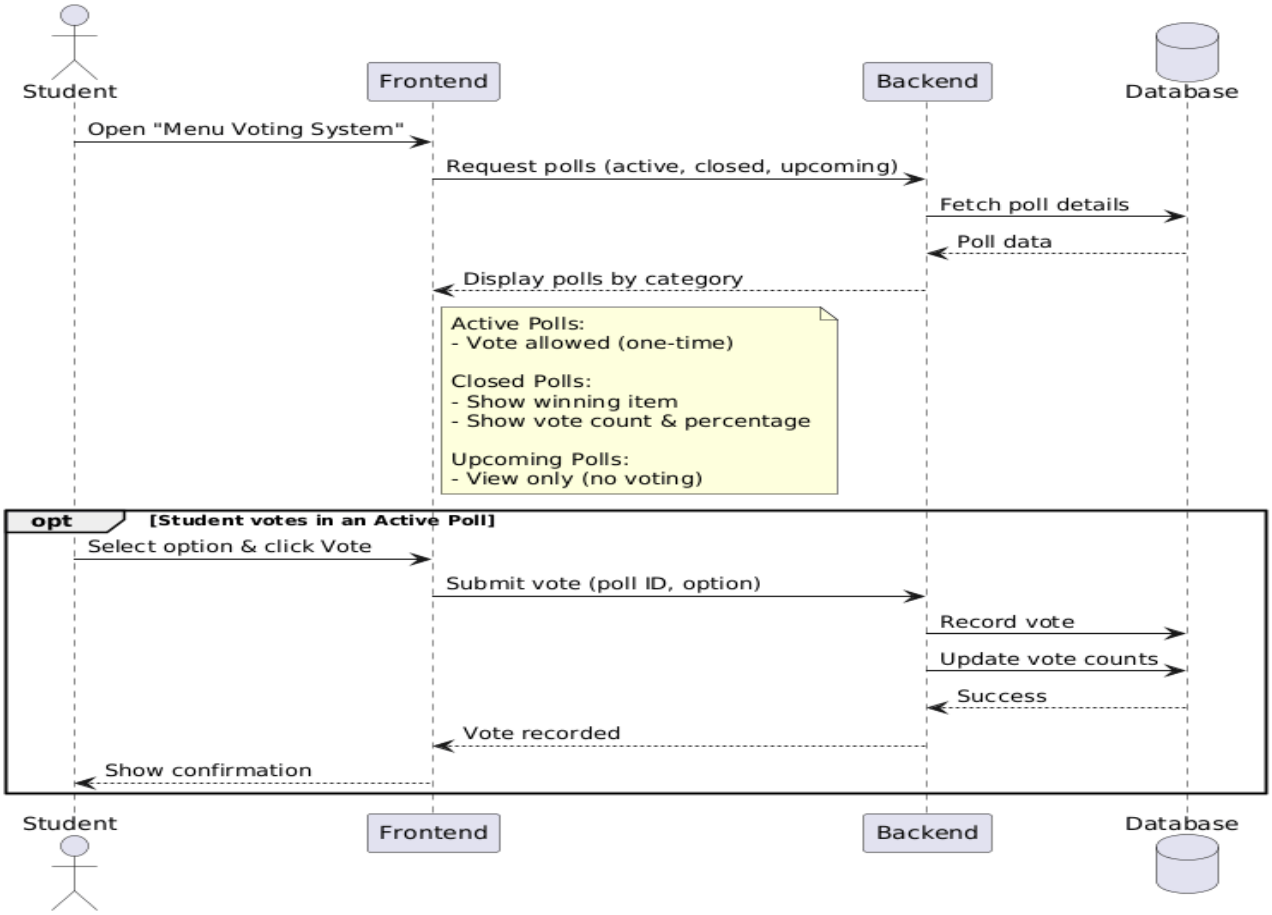
3.3.7 Rebate System:



3.3.8 View Dues and Transaction:

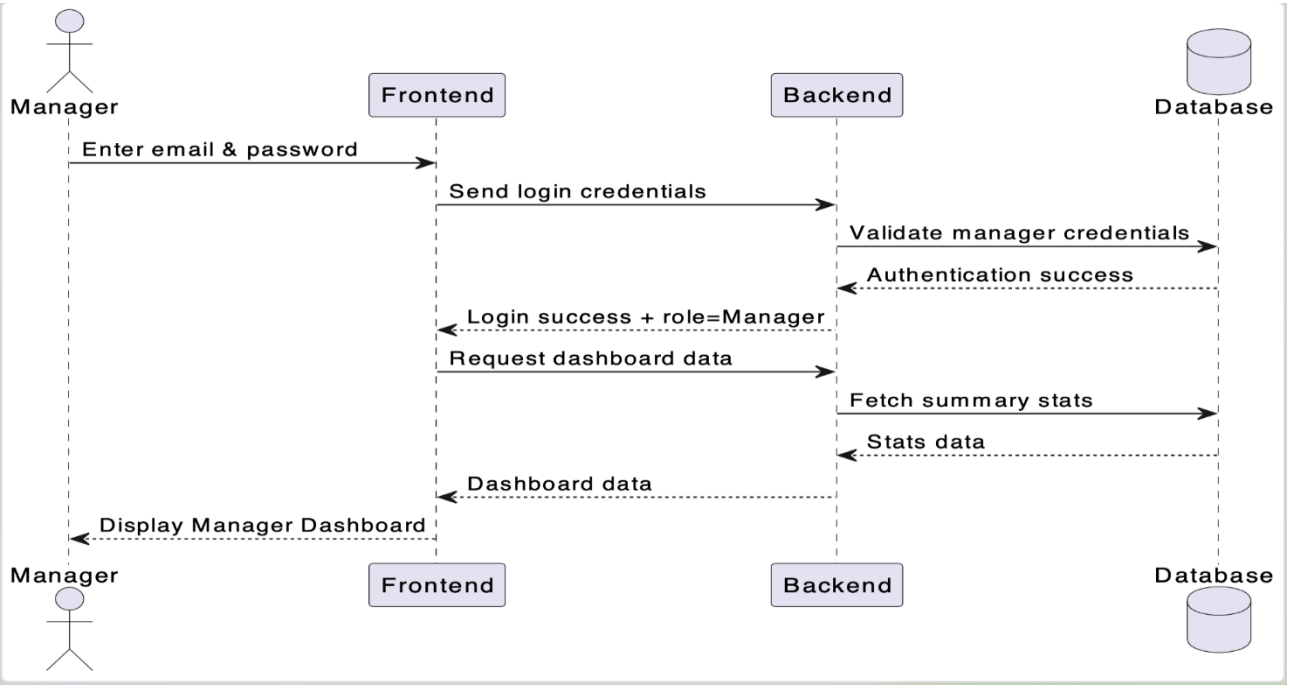


3.3.9 Voting:

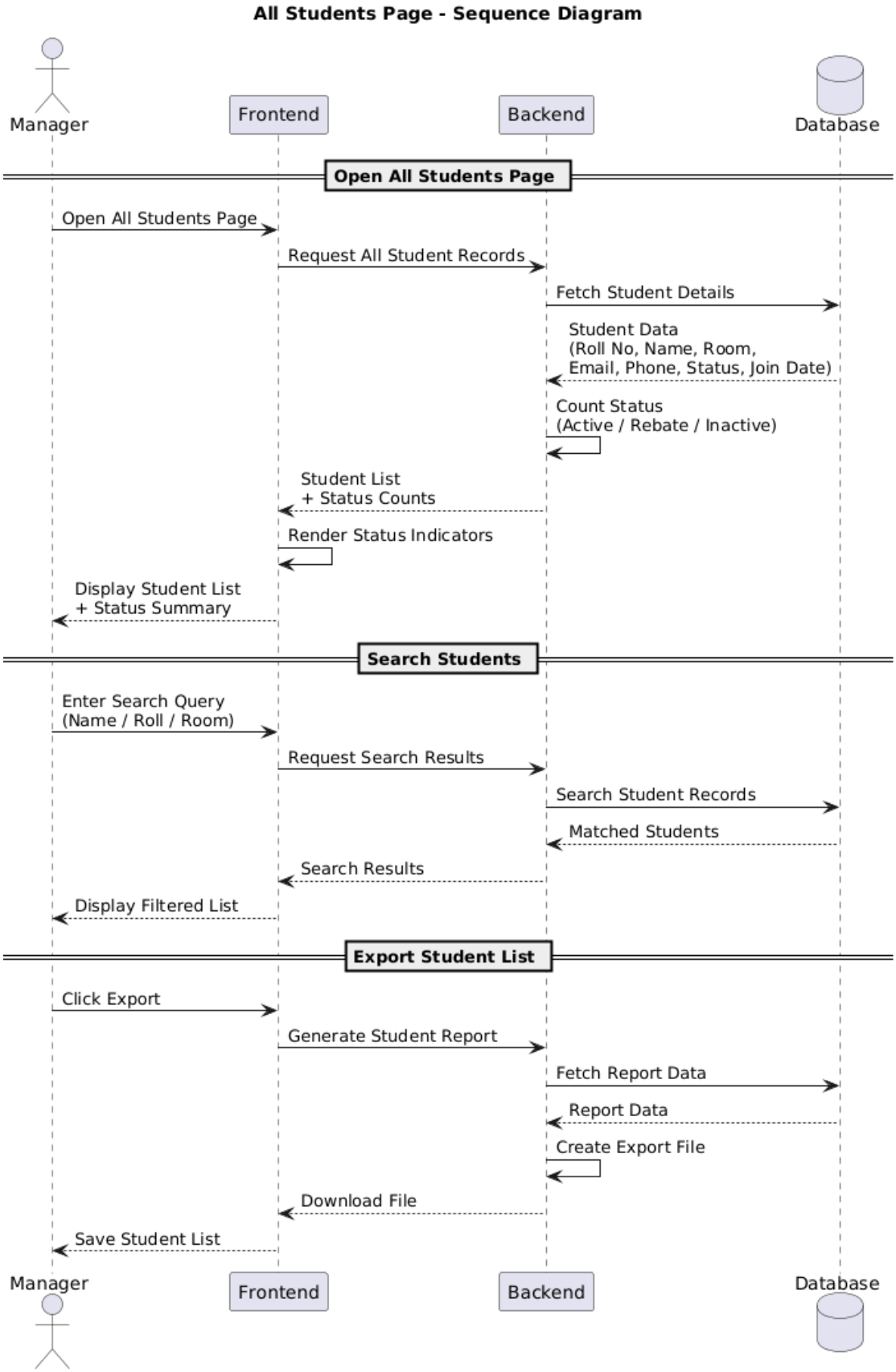


- **Manager Sequence Diagrams**

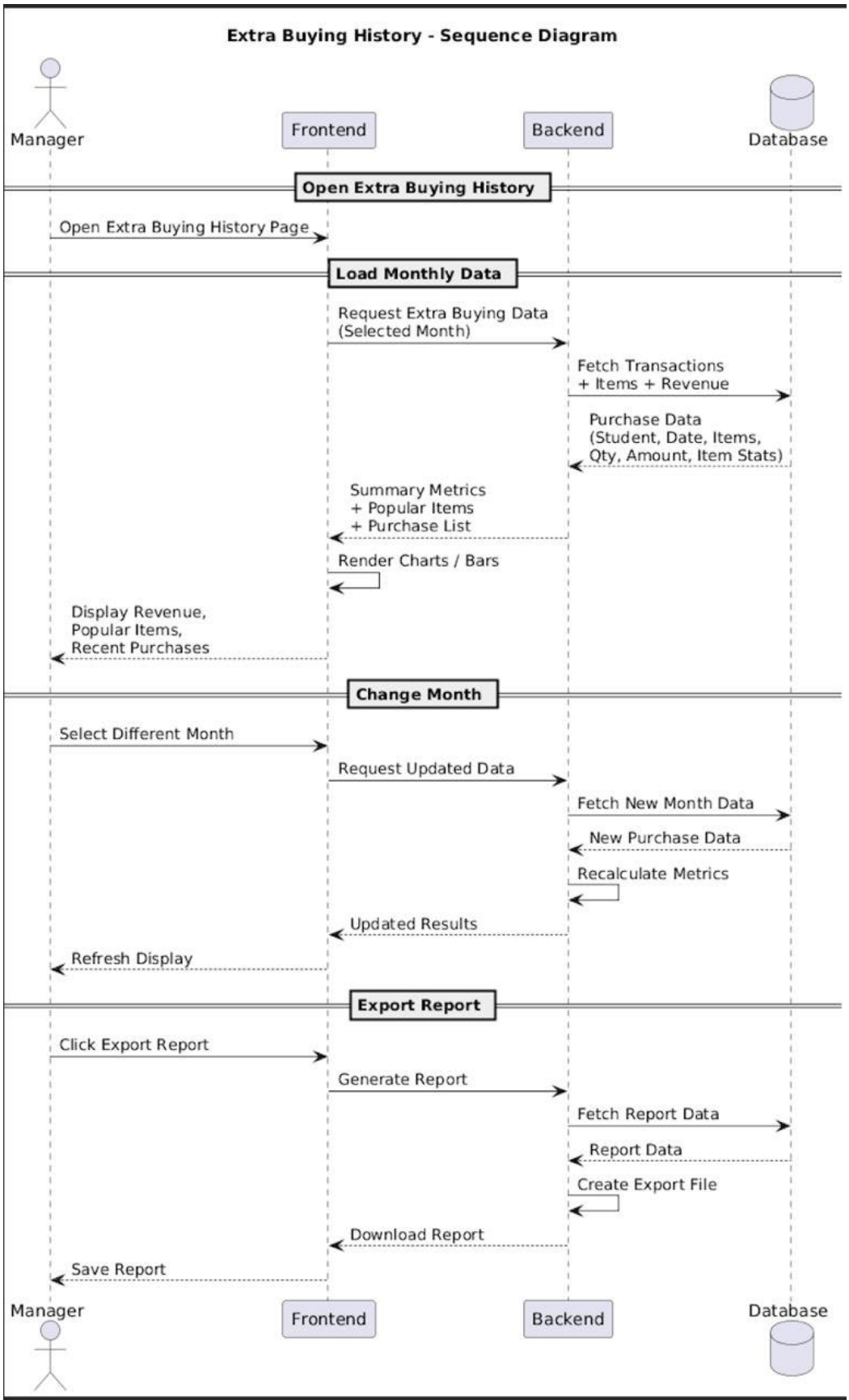
3.3.10 Dashboard:



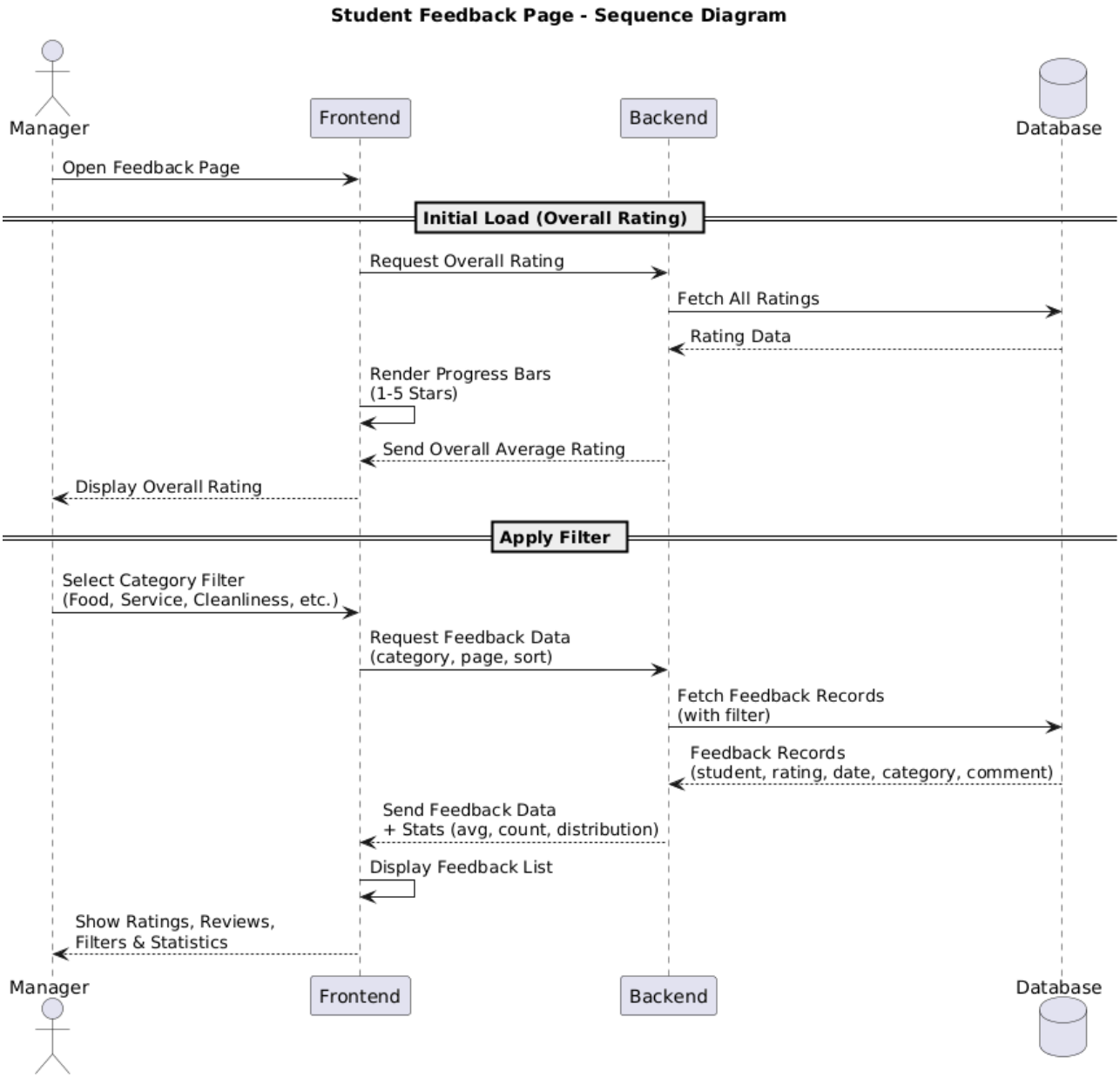
3.3.11 All Students page:



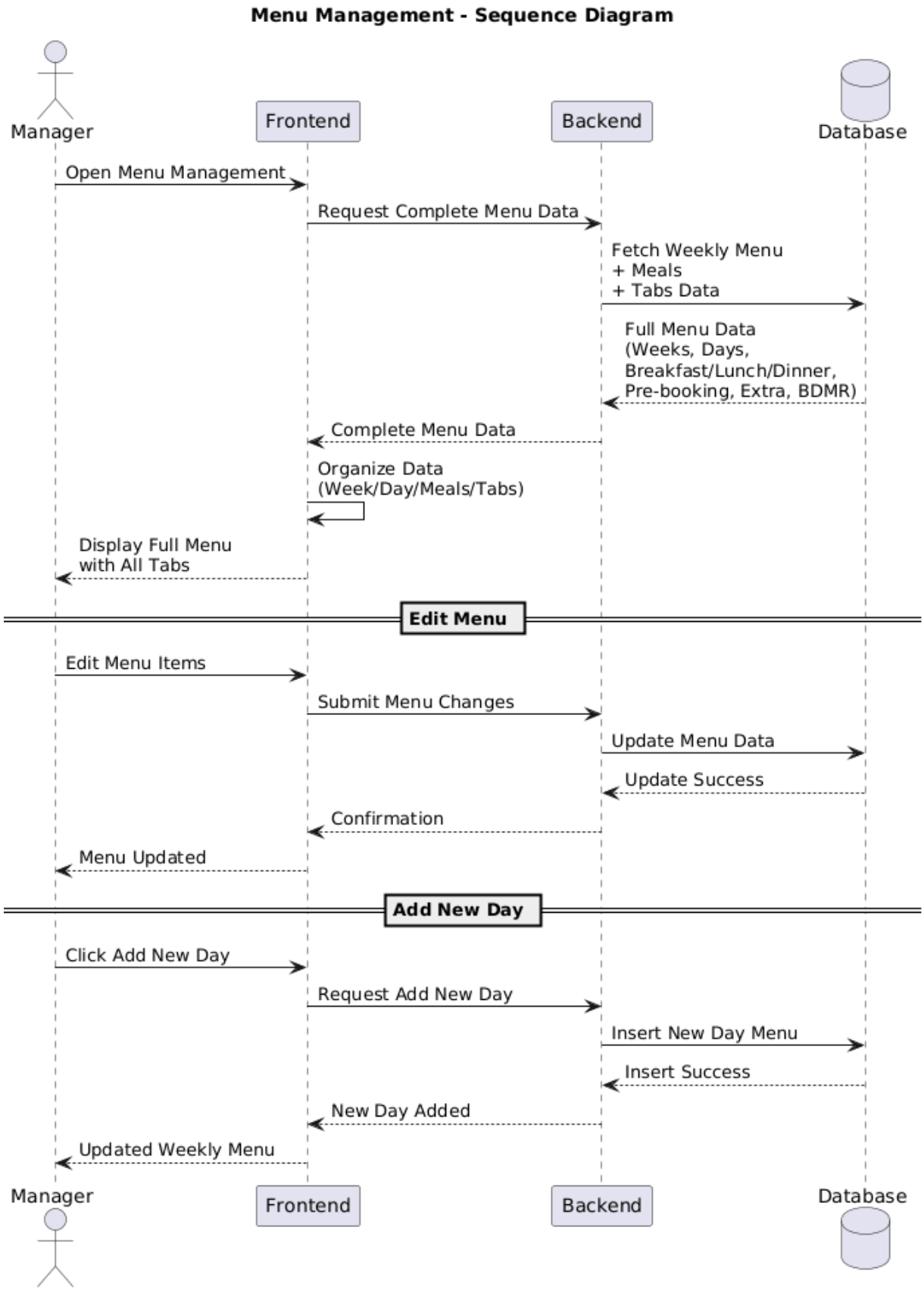
3.3.12 Extra Buying:



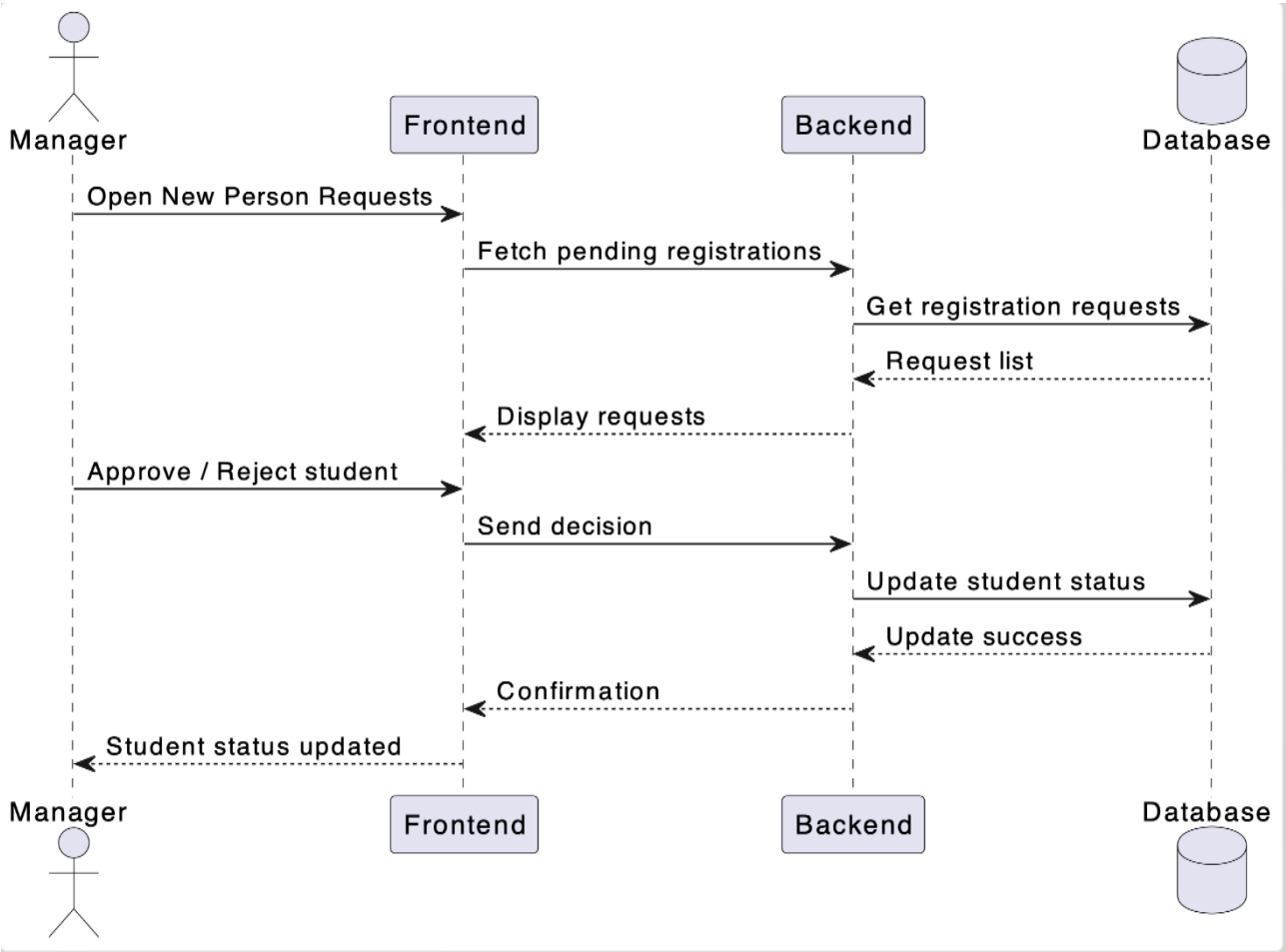
3.3.13 Feedback Page:



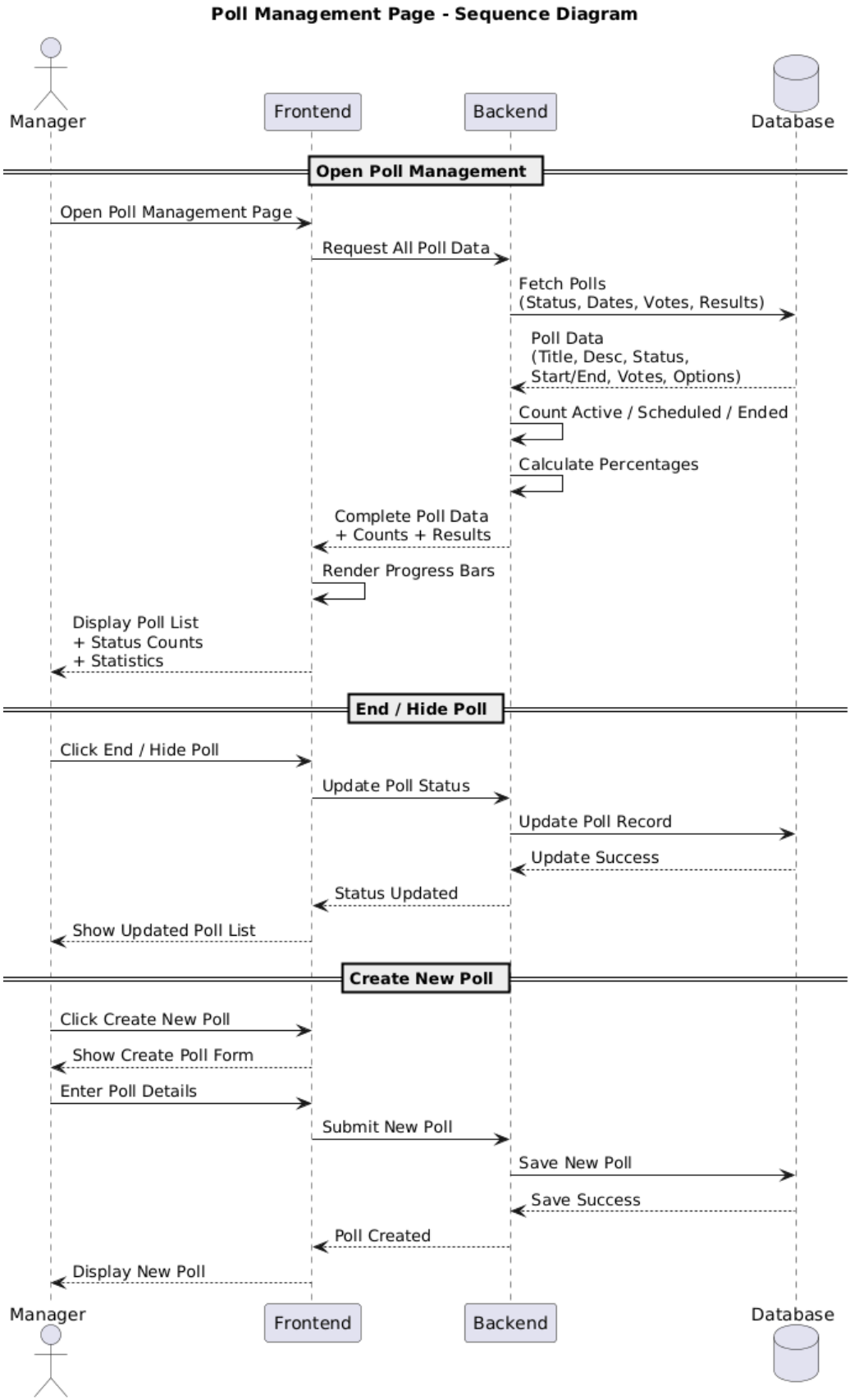
3.3.14 Mess Management:



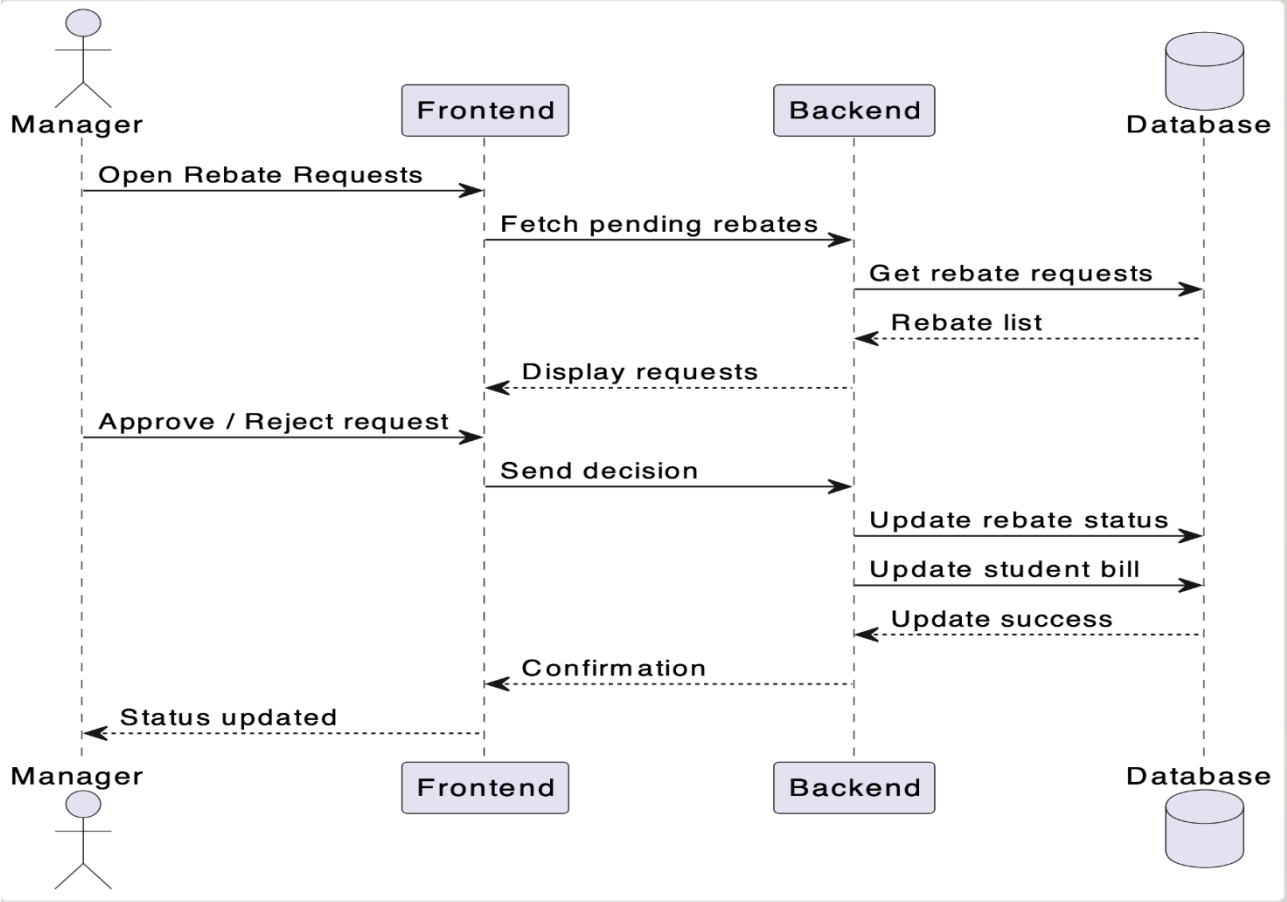
3.3.15 New Registration:



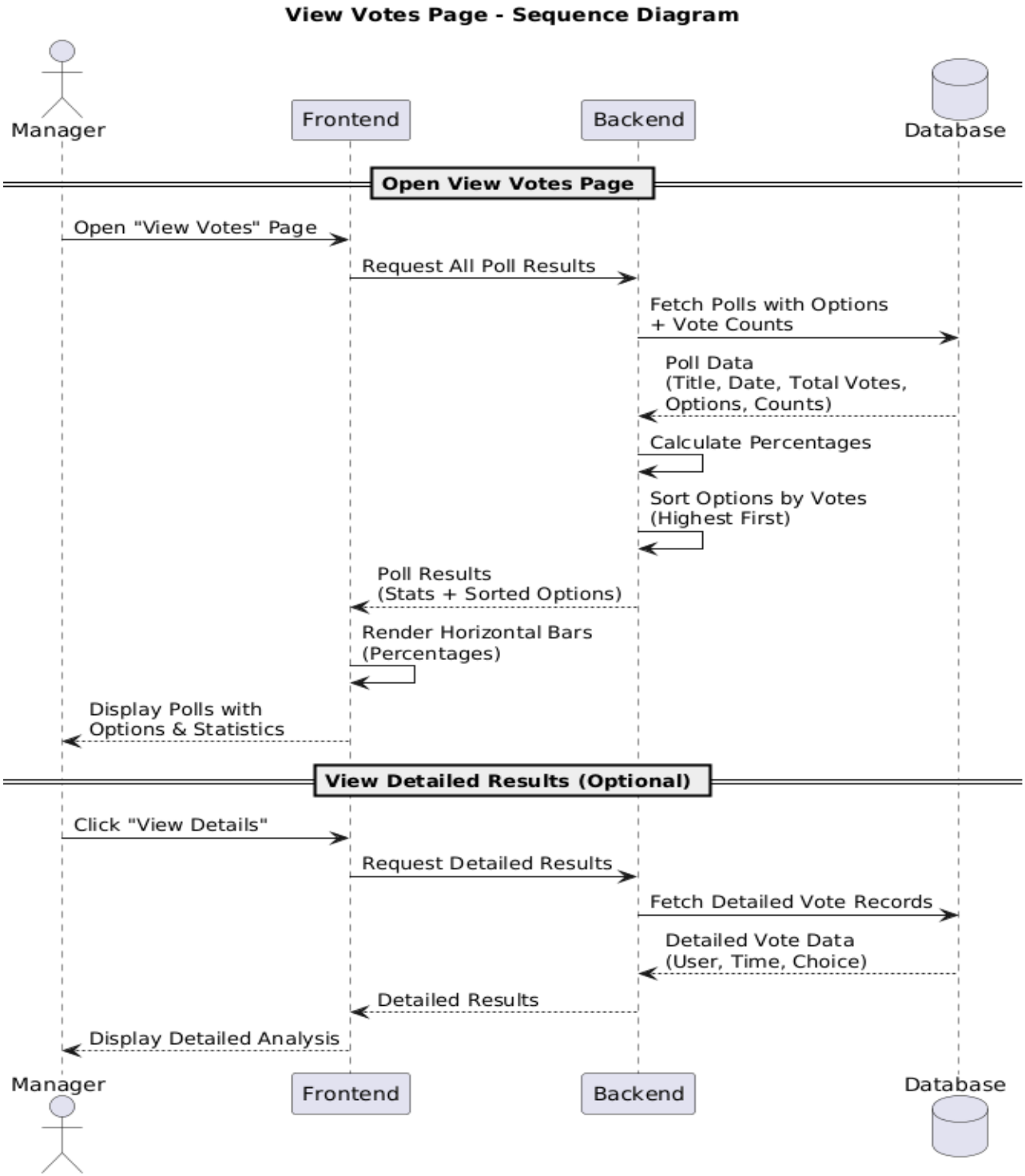
3.3.16 Poll Management:



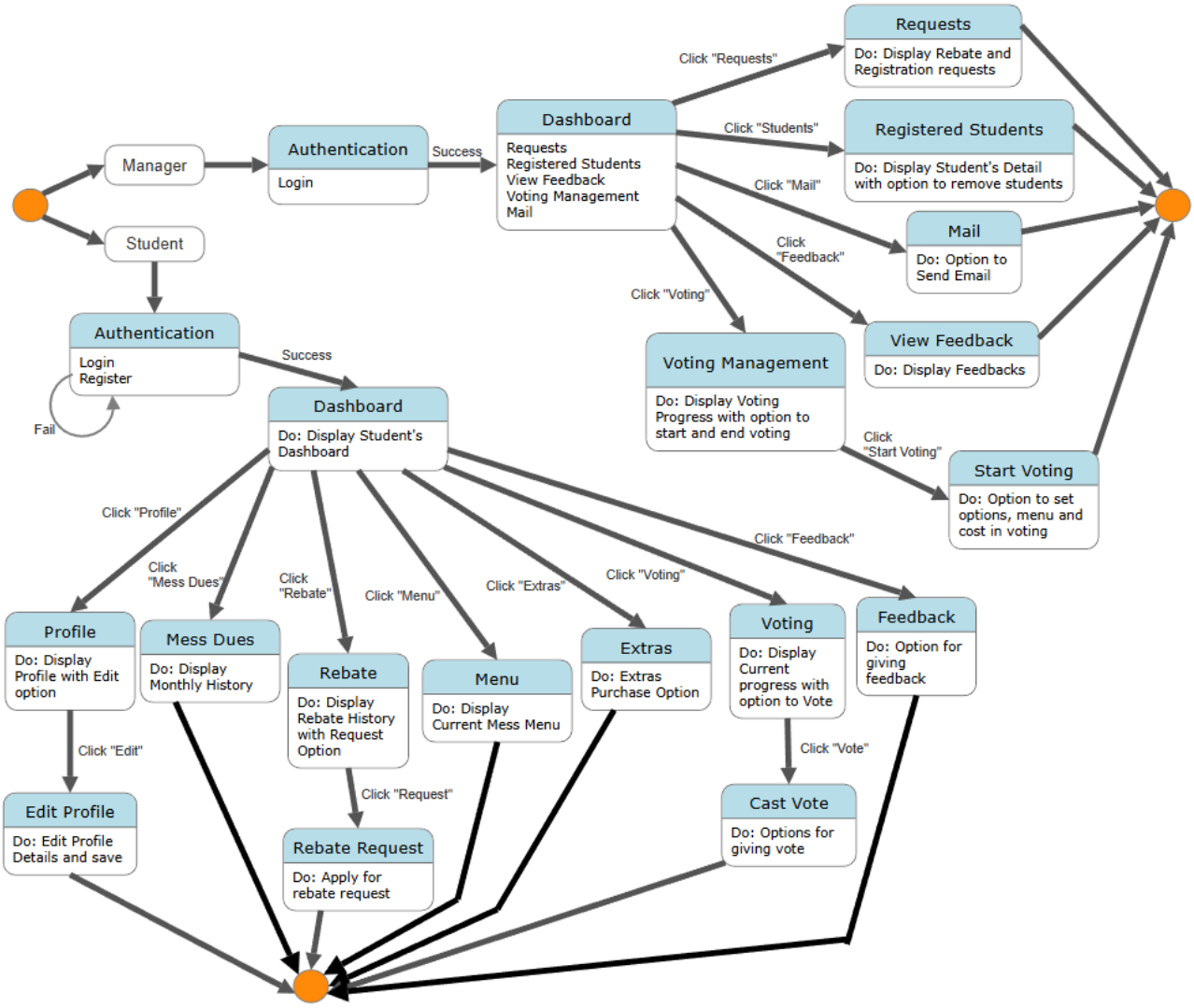
3.3.17 Rebate Management:



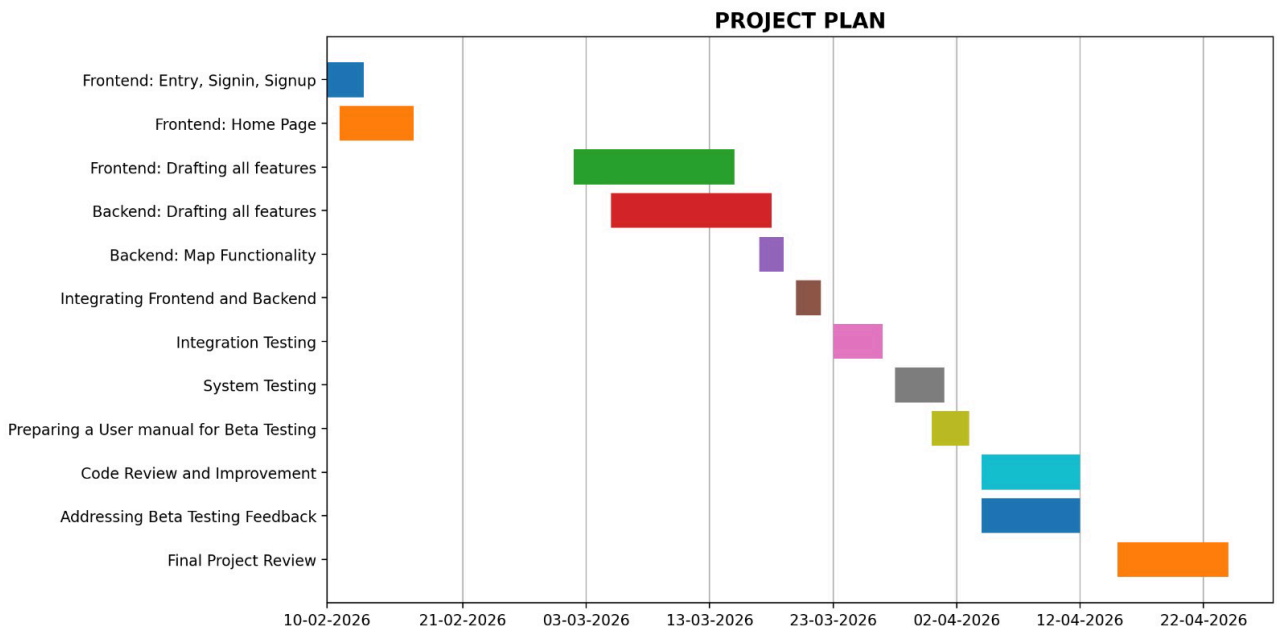
3.3.18 View Votes:



3.4 State Diagrams



4 Project Plan



START DATE	END DATE	TASK DESCRIPTION	DURATION (days)	TEAM MEMBERS
10-02-26	13-03-26	Frontend: Entry, Sign in, Signup	4	Full Team
11-02-26	17-03-26	Frontend: Home Page	7	Full Team
02-03-26	15-03-26	Frontend: Drafting all features	14	Full Team
05-03-26	18-03-26	Backend: Drafting all features	14	Full Team
17-03-26	19-03-26	Backend: Map Functionality	3	Full Team
20-03-26	22-03-26	Integrating Frontend and Backend	3	Full Team
23-03-26	27-03-26	Integration Testing	5	Full Team
28-03-26	01-04-26	System Testing	5	Full Team
31-03-26	03-04-26	Preparing User manual for Beta Testing	4	Full Team
04-04-26	12-04-26	Code Review and Improvement	9	Full Team

04-04-26	*12-04-26*	*Addressing Beta Testing Feedback*	*9*	*Full Team*
15-04-26	*24-04-26*	*Final Project Review*	*10*	*Full Team*

The following are the broad classification of the tasks:

Front End Design: *A front-end designer not only creates the aesthetics and wireframes for the product but also turns them into reality via code. Any component of the website which the end User is able to manipulate can be classified as Front End.*

Back End Development: *It is everything that the users don't see and contains behind-the-scenes activities that occur when performing any action on a website. It focuses primarily on databases, backend logic, APIs, and Servers.*

Appendix A - Group Log

- We have made a WhatsApp group for effective communication amongst ourselves.
- We have created a private repository on GitHub on which we would be regularly pushing our codes to maintain efficiency and collaboration.
- We have broadly divided the team into parts, all members would be working on different aspects of frontend and backend so that all members understand the mechanism of software development and are able to implement it.

The following is the group log for the development of this SDD (Software Design Document).

Date	Meet type	Final outcome of meet
27 January 2026	Online	Forming groups among our team to tackle different aspects of the project
30 January 2026	Offline	Dividing sections of the Software Design Document, and assigning them to members of the team
1 February 2026	Offline	To check on the progress of all team members and clarification of specific aspects of software.
5 February 2026	Online	Finalisation of Project timeline and other aspects.
6 February 2026	Offline	Final meet amongst the team members as a robust draft of the Software Design Document.